ROZDZIAŁ 2.

Nasz pierwszy projekt uczenia maszynowego

W tym rozdziale stworzymy od początku do końca przykładowy projekt uczenia maszynowego — będziemy udawać, że jesteśmy świeżo zatrudnionymi analitykami danych w agencji handlu nieruchomościami[1]. Aby nam się udało, musimy wykonać następujące czynności:

1. Przeanalizować całokształt czekającego nas zadania.
2. Uzyskać dane.
3. Odkryć i zwizualizować dane w celu rozpoznania wzorców i dodatkowych informacji.
4. Przygotować dane pod względem algorytmów uczenia maszynowego.
5. Wybrać i wyuczyć model.
6. Dostroić model.
7. Zaprezentować rozwiązanie.
8. Uruchomić, monitorować i utrzymywać system.

Praca z rzeczywistymi danymi

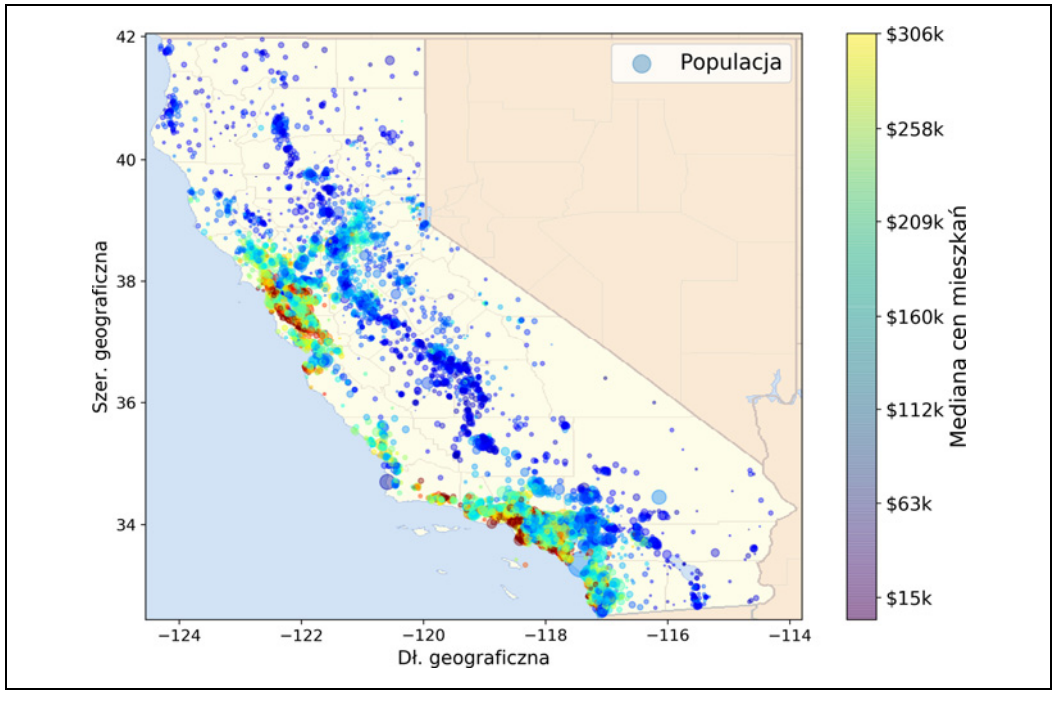
Podczas poznawania zagadnień uczenia maszynowego najlepiej jest eksperymentować na rzeczywistych, a nie sztucznych danych. Na szczęście są dostępne tysiące otwartych zbiorów danych, opisujące chyba wszelkie możliwe dziedziny życia. Poniżej wymieniam kilka miejsc, w których możesz szukać interesujących Cię danych:

- popularne, otwarte repozytoria danych:
  - repozytorium `uczenia` maszynowego Uniwersytetu Kalifornijskiego w Irvine (*http://archive.ics.uci.edu/ml/index.php*),
  - zbiory danych Kaggle (*https://www.kaggle.com/datasets*),
  - zbiory danych w usłudze Google AWS (*https://registry.opendata.aws/*),

[1] Projekt ten jest fikcyjny; jego zadaniem jest ukazanie poszczególnych etapów przygotowywania projektu uczenia maszynowego, a nie przekazywanie wiedzy na temat obrotu nieruchomościami.

- metaportale (zawierają one listy otwartych repozytoriów danych):
  - Data Portals (*http://dataportals.org/*),
  - OpenDataMonitor *(http://opendatamonitor.eu/*),
  - Quandl *(http://quandl.com/*),
- inne serwisy zawierające listy popularnych otwartych repozytoriów danych:
  - lista zbiorów danych dotyczących uczenia maszynowego na Wikipedii (*https://en.wikipedia.org/wiki/List_of_datasets_for_machine_learning_research*),
  - Quora.com (*https://www.quora.com/Where-can-I-find-large-datasets-open-to-the-public*),
  - temat na forum Reddit (*https://www.reddit.com/r/datasets/*).

Na użytek tego rozdziału wykorzystamy zestaw danych California Housing Prices (w wolnym tłumaczeniu „ceny domów w Kalifornii"), stanowiący część repozytorium StatLib[2] (rysunek 2.1). Dane te zostały opracowane na podstawie spisu ludności Kalifornii z 1990 roku. Nie są one już zbyt aktualne (w tamtym czasie można było nabyć dom w Bay Area za całkiem rozsądne pieniądze), ale pod wieloma względami mają one charakter edukacyjny, dlatego będziemy udawać, że są całkiem świeże. W celach dydaktycznych dodałem również atrybut kategorialny i usunąłem kilka cech.



Rysunek 2.1. *Ceny domów w Kalifornii*

[2] Pierwotnie zbiór tych danych pojawił się w artykule R. Kelleya Pace'a i Ronalda Barry'ego *Sparse Spatial Autoregressions*, „Statistics & Probability Letters 33", nr 3 (1997), s. 291 – 297.

Przeanalizuj całokształt projektu

Witaj w Korporacji Inteligentne Nieruchomości! Twoim pierwszym zadaniem jest wykorzystanie danych spisu ludności Kalifornii do stworzenia modelu cen mieszkań w tym stanie. Dane te zawierają takie informacje jak wielkość populacji, mediana dochodów, mediana cen mieszkań — wszystko rozdzielone pomiędzy poszczególne grupy bloków Kalifornii. Grupami bloków nazywamy najmniejsze jednostki terytorialne, dla których są publikowane próbki danych ze spisu ludności (zazwyczaj na typową grupę bloków składa się od 600 do 3000 osób). Dla uproszczenia będziemy nazywali je „dystryktami".

Twój model powinien uczyć się z tych danych i przy użyciu pozostałych metryk być w stanie przewidywać medianę cen mieszkań w dowolnym dystrykcie.



Skoro jesteśmy poukładanymi analitykami danych, naszą pierwszą czynnością powinno być przygotowanie listy kontrolnej projektu. Na początek możesz skorzystać z listy umieszczonej w dodatku B; powinna się ona nadawać do większości projektów uczenia maszynowego, nie zapomnij jednak dostosować jej do własnych potrzeb. W niniejszym rozdziale zajmiemy się wieloma elementami umieszczonymi na tej liście, ale też pominiemy niektóre, gdyż albo nie wymagają objaśnienia, albo zostaną omówione w następnych rozdziałach.

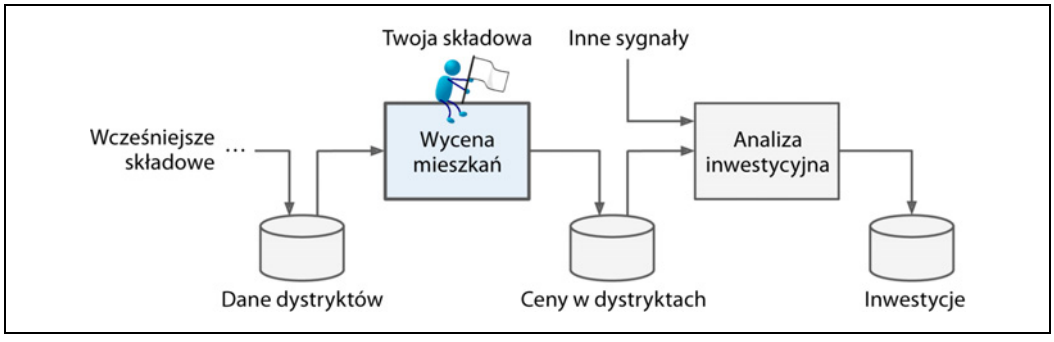
Określ zakres problemu

Pierwszym pytaniem, jakie należałoby zadać przełożonemu, byłoby: „Jaki jest dokładny cel biznesowy?"; utworzenie modelu prawdopodobnie nie jest ostatecznym celem samym w sobie. W jaki sposób firma zamierza skorzystać i zarobić na tym modelu? Znajomość celu jest istotna, ponieważ pomaga określić zakres problemu, dobrać odpowiednie algorytmy oraz miarę wydajności służącą do oceny wydajności modelu, a także pozwala oszacować wysiłek potrzebny do zoptymalizowania działania modelu.

Twój szef odpowiada, że wynik tego modelu (prognoza mediany cen mieszkań w danym dystrykcie) zostanie przekazany do innego systemu uczenia maszynowego (rysunek 2.2) wraz z wieloma innymi sygnałami[3]. Dzięki takiemu przepływowemu systemowi możliwe staje się określenie, czy warto inwestować na danym obszarze. Wszystko musi być dopięte na ostatni guzik, gdyż uzyskiwane rezultaty będą miały bezpośredni wpływ na obroty.

Następnie należałoby zapytać szefa, jak wygląda obecne rozwiązanie (jeśli w ogóle jakieś istnieje). Często możesz uzyskać w ten sposób dostęp do referencyjnej wydajności, a także podpowiedzi ułatwiające rozwiązanie problemu. Twój szef odpowiada, że obecnie ceny domów są szacowane ręcznie przez zespół ekspertów: gromadzą oni aktualne informacje o dystryktach, a gdy nie są w stanie wyliczyć mediany cen mieszkań, korzystają ze skomplikowanych reguł.

[3] Zgodnie z teorią informacji, którą Claude Shannon, zdefiniował w firmie Bell Labs w celu poprawy jakości telekomunikacji, **sygnałem** nazywamy informację przekazywaną systemowi uczenia maszynowego. Zgodnie z tą teorią zależy nam na uzyskaniu wysokiej wartości stosunku sygnału do szumu.



Rysunek 2.2. *Potok uczenia maszynowego używany w branży nieruchomości*

Jest to bardzo kosztowna i czasochłonna czynność, a uzyskane tą metodą oszacowania nie są wcale takie dokładne; często w sytuacjach, gdy analitykom udaje się wyliczyć medianę cen mieszkań, okazuje się, że pomylili się w oszacowaniach nawet o ponad 20%. Dlatego zarząd firmy uważa, że warto byłoby wyuczyć model przewidujący medianę cen mieszkań w danym dystrykcie za pomocą innych danych opisujących dany dystrykt. Znakomitym źródłem potrzebnych informacji okazuje się spis ludności, ponieważ możemy w nim znaleźć medianę cen mieszkań wyliczoną dla tysięcy dystryktów, a także wiele innych danych.

Potoki

Potokiem (ang. *pipeline*) danych nazywamy sekwencję **składowych** przetwarzanych danych. Potoki są bardzo popularne w systemach uczenia maszynowego, ponieważ manipulujemy w nich olbrzymią ilością danych oraz przeprowadzamy na nich wiele przekształceń.

Składowe są zazwyczaj przetwarzane asynchronicznie. Każda składowa pobiera znaczną ilość danych, przetwarza je i umieszcza wyniki w innym magazynie informacji. Po pewnym czasie te przetworzone wyniki są pobierane przez następną składową potoku, w której zostają znowu przetworzone i umieszczone w kolejnym magazynie danych. Każda składowa jest w znacznej mierze uniezależniona od pozostałych; jedynym interfejsem pomiędzy poszczególnymi składowymi jest magazyn danych. W ten sposób uzyskujemy prosty do zrozumienia system (za pomocą wykresów graficznych), a poszczególne zespoły mogą skoncentrować się na własnych składowych. Poza tym awaria danej składowej nie wpływa na pracę następnych składowych (przynajmniej przez pewien czas), gdyż mogą one korzystać z ostatniego prawidłowego wyniku wygenerowanego przez uszkodzoną składową. Dzięki temu taka architektura jest dość odporna.

Z drugiej strony, przy braku odpowiednich mechanizmów monitorowania uszkodzona składowa może przez dłuższy czas niezauważona dostarczać nieprawidłowe wyniki. Dane stają się nieaktualne i spada ogólna wydajność systemu.

Po uzyskaniu tych informacji możesz zająć się projektowaniem systemu. Najpierw zdefiniujmy problem: rozwiążemy go za pomocą uczenia nienadzorowanego, nadzorowanego czy może przez wzmacnianie? Jest to zadanie klasyfikacji, regresji czy jeszcze jakieś inne? Powinniśmy korzystać z technik uczenia przyrostowego czy wsadowego? Zanim przejdziemy dalej, przerwij na chwilę czytanie i postaraj się samodzielnie odpowiedzieć na te pytania.

Masz już odpowiedzi? Zastanówmy się: z pewnością mamy tu do czynienia z klasycznym zadaniem uczenia nadzorowanego, ponieważ będziemy korzystać z **oznakowanych** przykładów uczących (każdy

przykład ma zdefiniowany od razu oczekiwany wynik, np. medianę cen mieszkań w dystrykcie). Jest to również klasyczne zadanie regresyjne, ponieważ musimy przewidzieć jakąś wartość. Mówiąc dokładniej, stajemy przed problemem **regresji wielorakiej** (ang. *multiple regression*), gdyż do prognozowania wyniku nasz system wykorzysta wiele cech (populację dystryktu, medianę dochodów itd.). Możemy także traktować ten problem jako zadanie **regresji jednoczynnikowej** (ang. *univariate regression*), ponieważ dla każdego dystryktu staramy się przewidzieć tylko pojedynczą wartość. Gdybyśmy próbowali prognozować wiele wartości dla każdego dystryktu, mielibyśmy do czynienia z **regresją wieloczynnikową** (ang. *multivariate regression*). Na koniec zwróćmy uwagę, że dane nie będą dostarczane w ciągły sposób, nie będziemy musieli dynamicznie dostosowywać systemu do zmieniających się danych, a same dane są wystarczająco małe, aby zmieścić się w pamięci, dlatego powinna nam wystarczyć zwykła, wsadowa metoda uczenia.



Gdyby dane miały jednak ogromne rozmiary, mogłabyś/mógłbyś rozdzielić czynności uczenia wsadowego pomiędzy kilka serwerów (przy użyciu techniki MapReduce) albo zastąpić je po prostu mechanizmem uczenia przyrostowego.

Wybierz metrykę wydajności

Kolejnym etapem jest dobór metryki wydajności. W przypadku zagadnień regresyjnych klasyczną miarą wydajności jest **pierwiastek błędu średniokwadratowego** (ang. *Root Mean Square Error* — RMSE). Dowiadujemy się dzięki niemu, w jakim stopniu model myli się w przewidywaniach — wraz ze wzrostem wartości błędu rośnie również waga tej metryki. Równanie 2.1 przedstawia wzór matematyczny używany do wyliczania błędu RMSE.

Równanie 2.1. Pierwiastek błędu średniokwadratowego (RMSE)

$$\mathrm{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m}\sum_{i=1}^{m}\left(h\left(\mathbf{x}^{(i)}\right) - y^{(i)}\right)^2}$$

Notacje

Powyższy wzór zawiera kilka najpowszechniejszych notacji używanych w uczeniu maszynowym, z których będziemy korzystać w dalszej części książki:

- Wartość m określa liczbę elementów zbioru uczącego, wobec których będziemy mierzyć błąd RMSE,
  - przykładowo, jeśli chcesz obliczyć błąd RMSE dla zbioru walidacyjnego 2000 dystryktów, to $m = 2000$.
- Wartość $\boldsymbol{x}^{(i)}$ stanowi wektor wartości wszystkich cech (z pominięciem etykiet) i-tego przykładu, natomiast $y^{(i)}$ stanowi etykietę tejże próbki (czyli pożądaną wartość wyniku dla danego przykładu),
  - na przykład, jeśli pierwszy dystrykt w zbiorze danych znajduje się na długości geograficznej –118,29°W i szerokości geograficznej 33,91°N, a do tego wiemy, że jego populacja wynosi 1416 mieszkańców przy średnim dochodzie 38 372 dolarów, natomiast mediana ceny mieszkań na tym obszarze wynosi 156 400 dolarów (na razie ignorujemy pozostałe cechy), to możemy zapisać te dane w następujący sposób:

$$\mathbf{x}^{(1)} = \begin{pmatrix} -118,29 \\ 33,91 \\ 1416 \\ 38372 \end{pmatrix}$$

i

$y^{(1)}=156400.$

- Wartość ***X*** oznacza macierz zawierającą wartości wszystkich cech (z pominięciem etykiet) wszystkich przykładów składających się na zbiór danych. Dla każdego przykładu przeznaczony jest jeden wiersz, natomiast *i*-ty wiersz stanowi transpozycję wektora $\boldsymbol{x}^{(i)}$; zwróć uwagę na zapis $(\boldsymbol{x}^{(i)})^T$,[4]
  - na przykład zgodnie z podanym powyżej opisem pierwszego dystryktu macierz ***X*** będzie wyglądała następująco:

$$\mathbf{X} = \begin{pmatrix} (\mathbf{x}^{(1)})^T \\ (\mathbf{x}^{(2)})^T \\ \vdots \\ (\mathbf{x}^{(1999)})^T \\ (\mathbf{x}^{(2000)})^T \end{pmatrix} = \begin{pmatrix} -118,29 & 33,91 & 1416 & 38372 \\ \vdots & \vdots & \vdots & \vdots \end{pmatrix}$$

- Parametr *h* jest funkcją prognozującą naszego systemu (bywa ona zwana także **hipotezą**). Po wprowadzeniu wartości wektora cech $\boldsymbol{x}^{(i)}$ do systemu zostaje wygenerowana prognozowana wartość $\hat{y}^{(i)} = h(\boldsymbol{x}^{(i)})$ dla tego przykładu (symbol $\hat{y}$ nazywamy „igrekiem z daszkiem”),
  - przykładowo, jeśli Twój system wyliczy, że mediana cen mieszkań w pierwszym dystrykcie wynosi 158 400 dolarów, to $\hat{y}^{(1)} = h(\boldsymbol{x}^{(1)}) = 158400$. Wartość błędu predykcyjnego dla tego dystryktu to $\hat{y}^{(1)} - y^{(1)} = 2000$.
- Zapis $RMSE(\mathbf{X},h)$ oznacza funkcję kosztu mierzoną dla zbioru uczącego za pomocą hipotezy *h*.

Wartości skalarne (np. *m* lub $y^{(i)}$), a także nazwy funkcji (np. *h*) zapisujemy małymi literami i kursywą, symbole pogrubione (np. $\boldsymbol{x}^{(i)}$) oznaczają wektory, natomiast duże, pogrubione litery (np. ***X***) są zarezerwowane dla macierzy.

Pomimo że preferowaną metryką wydajności w zadaniach regresyjnych jest pomiar błędu RMSE, w pewnych sytuacjach lepiej sprawdza się inna funkcja. Załóżmy na przykład, że w zestawie danych znajduje się wiele dystryktów odstających od reszty. W takim przypadku warto byłoby wyliczyć **średni absolutny błąd** (ang. *Mean Absolute Error* — MAE; zwany jest także średnim odchyleniem bezwzględnym, ang. *Average Absolute Deviation*) (równanie 2.2).

Równanie 2.2. Średni absolutny błąd (MAE)

$$MAE(\mathbf{X},h) = \frac{1}{m}\sum_{i=1}^{m}\left|h\left(\mathbf{x}^{(i)}\right) - y^{(i)}\right|$$

[4] Przypominam, że operator transpozycji przekształca wiersze na kolumny i odwrotnie.

Obydwie miary (RMSE i MAE) pozwalają na obliczanie odległości pomiędzy dwoma wektorami: wektorem prognoz i wektorem wartości docelowych. Dostępne są różne miary odległości (tzw. **normy**):

- Obliczenie pierwiastka błędu średniokwadratowego (RMSE) wiąże się z **normą euklidesową** — stanowi ona doskonale nam znaną notację odległości. Zwana jest ona także **normą ℓ_2** i jest zapisywana w postaci wyrażenia $\|\cdot\|_2$ (lub po prostu $\|\cdot\|$).
- Obliczenie średniego absolutnego błędu (MAE) wiąże się z **normą ℓ_1**, zapisywaną jako wyrażenie $\|\cdot\|_1$. Jest ona nazywana również **normą taksówkową**, **miejską** oraz **Manhattan**, ponieważ mierzy ona odległość pomiędzy dwoma punktami w mieście, jedynie poruszając się wzdłuż ortogonalnych bloków miasta.
- W ogólniejszym ujęciu **normę ℓ_k** wektora $\boldsymbol{v}$ zawierającego n elementów definiujemy następująco: $\|\mathbf{v}\|_k = \left(|v_0|^k + |v_1|^k + \cdots + |v_n|^k\right)^{\frac{1}{k}}$. Norma ℓ_0 podaje liczbę niezerowych elementów w danym wektorze, natomiast ℓ_∞ wyznacza w nim maksymalną wartość bezwzględną.
- Im wyższy indeks normy, tym bardziej skupia się ona na dużych wartościach i ignoruje małe wartości. Z tego właśnie powodu metryka RMSE jest bardziej wyczulona na elementy odstające niż miara MAE. Jednakże jeśli liczba tych elementów wykładniczo maleje (np. w przypadku krzywej dzwonowej), pomiary RMSE znakomicie się sprawdzają i generalnie jest zalecane używanie tej miary.

Sprawdź założenia

Przed przystąpieniem do tworzenia modelu zawsze warto sporządzić listę dotychczasowych założeń i je zweryfikować (przez Ciebie lub kogoś innego) — może Ci to pomóc już na wczesnym etapie w wychwyceniu jakichś problemów. Na przykład prognozowane przez Twój system ceny w poszczególnych dystryktach będą przekazywane do następnej składowej potoku, a Ty zakładasz, że będą one tam używane w zdefiniowanym przez nas formacie. A jeśli okaże się, że ceny te będą kategoryzowane (np. „tanie", „średnie" i „drogie") i dopiero tego typu wartości będą wykorzystywane? W takim wypadku uzyskanie dokładnej ceny mija się zupełnie z celem; Twój system musi jedynie prawidłowo przewidywać kategorie. Skoro tak, zagadnienie, którym się zajmujesz, powinno zostać oznaczone jako problem klasyfikacyjny, a nie regresyjny. Raczej nie chciałabyś/chciałbyś tego odkryć dopiero po kilkumiesięcznej pracy nad systemem regresyjnym.

Na szczęście po rozmowie z szefostwem i pozostałymi zespołami masz pewność, że potrzebne będą rzeczywiste ceny, a nie kategorie. Znakomicie! Wszystko przygotowane, masz zielone światło i możesz zabrać się za pisanie kodu!

Zdobądź dane

Czas pobrudzić sobie ręce. Bez wahania uruchom laptopa i w miarę zapoznawania się z treścią książki samodzielnie sprawdzaj omawiane przykłady, zawarte w notatnikach Jupyter. Wszelkie potrzebne materiały znajdziesz pod adresem *ftp://ftp.helion.pl/przyklady/uczem2.zip*.

Stwórz przestrzeń roboczą

W pierwszej kolejności należy zainstalować środowisko Python. Prawdopodobnie masz już ten etap za sobą. Jeśli nie, wszelkie potrzebne pliki znajdziesz na stronie *https://www.python.org/* [5].

Teraz musimy stworzyć katalog roboczy, w którym będziemy przechowywać kod projektu oraz zbiory danych. Otwórz terminal i wpisz następujące polecenia (po znaku zachęty *$*):

```
$ export ML_PATH="$HOME/um" # Jeśli chcesz, możesz zmienić nazwę katalogu
$ mkdir -p $ML_PATH
```

Potrzebnych nam będzie kilka modułów środowiska Python: Jupyter, NumPy, pandas, Matplotlib i Scikit-Learn. Jeśli już masz je zainstalowane, możesz spokojnie przejść do punktu „Pobierz dane". W przeciwnym razie istnieje wiele różnych sposobów ich instalacji (wraz z potrzebnymi zależnościami). Możesz użyć systemowego menedżera pakietów (np. *apt-get* w dystrybucji Ubuntu albo *MacPorts* lub *Homebrew* w systemie MacOS), zainstalować interfejs naukowy platformy Python (np. *Anaconda*) i skorzystać z jego zasobów albo po prostu wykorzystać wewnętrzny menedżer pakietów Python (*pip*), domyślnie instalowany (począwszy od wersji Python 2.7.9)[6] wraz z plikami binarnymi omawianego języka. Za pomocą poniższej komendy możesz sprawdzić, czy menedżer *pip* jest zainstalowany na Twoim systemie:

```
$ python3 –m pip --version
pip 19.3.1 from [...]/lib/python3.7/site-packages (python 3.7)
```

Aby system obsługiwał możliwość instalowania modułów binarnych, musisz mieć zainstalowaną najnowszą wersję menedżera *pip*. W celu zaktualizowania modułu *pip* wpisujemy następujące polecenie (dokładna wersja modułu może być inna)[7]:

```
$ python3 –m pip install --user -U pip
Collecting pip
[...]
Successfully installed pip-19.3.1
```

Możesz teraz zainstalować wszystkie wymagane moduły wraz z zależnościami przy użyciu następującej prostej komendy `pip` (jeżeli nie korzystasz ze środowiska wirtualnego, musisz dodać opcję `--user` albo uprawnienia administratora):

```
$ python3 –m pip install -U jupyter matplotlib numpy pandas scipy scikit-learn
Collecting jupyter
Downloading https://[...]/jupyter-1.0.0-py2.py3-none-any.whl
Collecting matplotlib
[...]
```

[5] Zalecam korzystanie z najnowszego wydania środowiska Python 3. Platforma Python 2.7+ powinna również działać bez zarzutu, ale pamiętaj, że jest już przestarzała. Wszystkie główne biblioteki naukowe porzucają obsługę wersji 2. Pythona, dlatego należy przenieść się jak najszybciej do środowiska Python 3.

[6] Zaprezentuję proces instalacji za pomocą menedżera *pip* w konsoli systemu Linux lub macOS. Może pojawić się potrzeba zmodyfikowania tych poleceń na innych systemach operacyjnych. W przypadku użytkowników systemów z rodziny Windows zalecam zainstalowanie platformy *Anaconda*.

[7] Jeżeli chcesz zaktualizować moduł *pip* dla wszystkich użytkowników komputera, musisz usunąć opcję `--user` i wykonać komendę z poziomu administratora (np. umieszczając na początku wiersza komendę `sudo` w systemie macOS lub Linux).

Środowisko izolowane

Jeśli chcesz pracować w izolowanym środowisku (do czego gorąco namawiam, gdyż możesz pracować nad różnymi projektami bez ryzyka konfliktu pomiędzy różnymi wersjami bibliotek), zainstaluj moduł `virtualenv`[8] za pomocą następującego polecenia (również w tym przypadku, jeżeli chcesz, aby środowisko wirtualne zostało zainstalowane dla wszystkich użytkowników komputera, usuń człon `--user` i uruchom komendę przy użyciu uprawnień administratora):

```
$ python3 –m pip install --user -U virtualenv
Collecting virtualenv
[...]
Successfully installed virtualenv-16.7.7
```

Możesz teraz stworzyć izolowane środowisko Pythona:

```
$ cd $ML_PATH
$ python3 –m virtualenv moje_srod
Using base prefix '[...]'
New python executable in [...]/um/moje_srod/bin/python3
Also creating executable in [...]/um/moje_srod/bin/python
Installing setuptools, pip, wheel...done.
```

Teraz za każdym razem, gdy zechcesz uruchomić to środowisko, wystarczy otworzyć terminal i wpisać poniższe polecenia:

```
$ cd $ML_PATH
$ source moje_srod/bin/activate # W Linuksie lub macOS
$ .\moje_srod\Scripts\activate # W Windowsie
```

Aby wyłączyć to środowisko, wpisz polecenie **`deactivate`**. Przy włączonym środowisku izolowanym będą w nim instalowane wszelkie pakiety dostarczane poprzez menedżer *pip*; dostęp do nich będzie miała wyłącznie platforma Python (jeśli chcesz uzyskać dostęp również do pakietów systemowych, stwórz środowisko izolowane przy użyciu opcji `--system-site-packages`). Więcej informacji znajdziesz w dokumentacji modułu `virtualenv`.

Jeżeli stworzyłaś/stworzyłeś środowisko wirtualne, musisz zarejestrować je w środowisku Jupyter i nadać mu nazwę:

```
$ python3 -m  ipykernel install --user --name=python3
```

Włączmy teraz aplikację Jupyter:

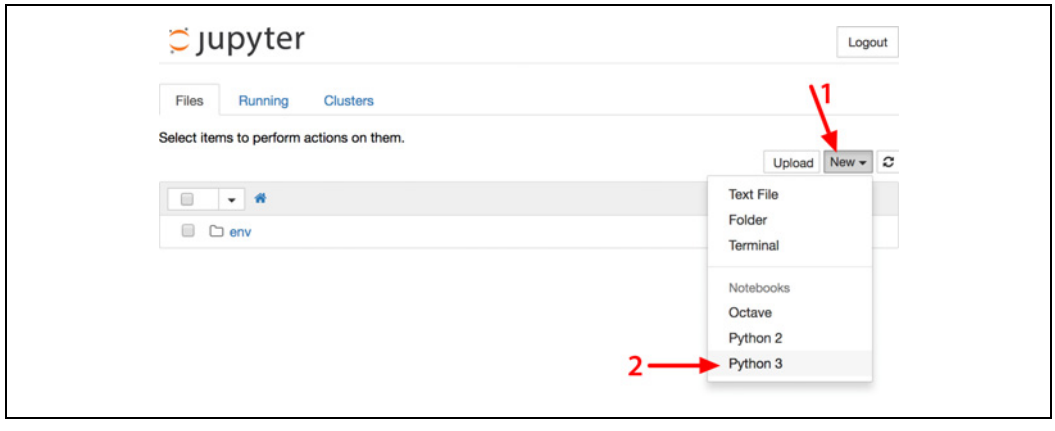
```
$ jupyter notebook
[…] Serving notebooks from local directory: [...]/um
[…] The Jupyter Notebook is running at:
http://localhost:8888/?token=60995e108e44ac8d8865a[...]
[…]  or http://127.0.0.1:8889/?token=60995e108e44ac8d8865a[...]
[…] Use Control-C to stop this server and shut down all kernels […]
```

Serwer Jupyter już działa w terminalu i nasłuchuje portu 888. Możesz zajrzeć na ten serwer, wpisując w przeglądarce adres *http://localhost:8888/* (zazwyczaj następuje to automatycznie w momencie

[8] Do alternatywnych narzędzi zaliczamy `venv` (bardzo przypomina `virtualenv` i stanowi część biblioteki standardowej), `virtualenvwrapper` (dodaje pewne funkcje do modułu `virtualenv`), `pyenv` (umożliwia łatwe przełączanie pomiędzy wersjami Pythona), a także `pipenv` (znakomite narzędzie autora m.in. popularnej biblioteki `requests`, stanowiące dodatek do modułów `pip` i `virtualenv`).

uruchomienia serwera). Twoim oczom powinien ukazać się pusty katalog roboczy (zawierający jedynie folder *env*, jeśli zainstalowałaś/zainstalowałeś również środowisko *virtualenv*).

Stworzymy teraz nowy notatnik Jupyter, klikając przycisk *New* i wybierając właściwą wersję środowiska Python[9] (rysunek 2.3). W ten sposób zostaje utworzony nowy notatnik, o nazwie *Untitled.ipynb*, w obszarze roboczym, następuje uruchomienie jądra Jupyter Python obsługującego tenże notatnik, a następnie notatnik ten zostaje otwarty w nowej zakładce. Zacznijmy od zmiany nazwy notatnika na *Mieszkania* (w rzeczywistości zostanie zmieniona nazwa pliku na *Mieszkania.ipynb*) — wystarczy kliknąć napis *Untitled* i wprowadzić własną nazwę.



Rysunek 2.3. *Twój obszar roboczy w aplikacji Jupyter*

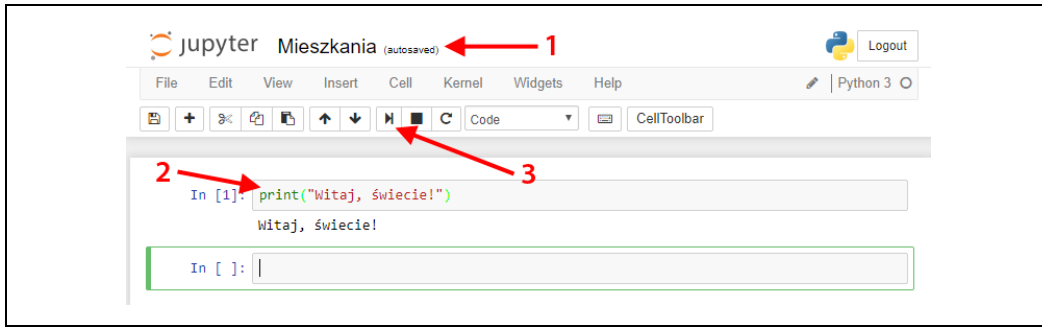
Notatnik składa się z komórek. W każdej komórce umieszczamy albo kod wykonywalny, albo sformatowany tekst. Obecnie w naszym notatniku znajduje się tylko jedna, pusta komórka, oznaczona jako *In [1]:*. Spróbuj wpisać w niej komendę `print(„Witaj, świecie!")` i kliknij przycisk odtwarzania (ikona oznaczona liczbą *3* na rysunku 2.4) lub użyj kombinacji klawiszy *Shift+Enter*. Zawartość tej komórki zostanie przesłana do jądra środowiska Python, gdzie nastąpi jej przetwarzanie, po czym ujrzymy jej wynik na ekranie. Rezultat zostaje umieszczony pod daną komórką, a ponieważ dotarłaś/dotarłeś do końca notatnika, automatycznie nastąpi utworzenie nowej komórki. Aby poznać podstawy obsługi notatników Jupyter, zapoznaj się z samouczkiem *User Interface Tour*, dostępnym w menu pomocy (*Help*) aplikacji Jupyter.

Pobierz dane

W typowym środowisku dane są najczęściej umieszczone w relacyjnej bazie danych (lub innym powszechnie używanym magazynie danych) i rozrzucone pomiędzy wiele tabel/dokumentów/plików. Aby uzyskać do nich dostęp, należy najpierw uzyskać autoryzację do ich przeglądania lub jakieś inne poświadczenia[10], a następnie zaznajomić się z używanym schematem danych. W naszym projekcie

[9] Zwróć uwagę, że aplikacja Jupyter obsługuje różne wersje języka Python, a nawet wiele innych języków, takich jak R czy Octave.

[10] Musisz również brać pod uwagę ograniczenia prawne, np. prywatne pola, których nie należy nigdy kopiować do niezbyt bezpiecznych magazynów danych.



Rysunek 2.4. *Witaj, świecie!*

jest jednak znacznie prościej: pobierzemy jedynie archiwum (*housing.tgz*) zawierające plik CSV (ang. *comma-separated values* — wartości rozdzielone przecinkami) o nazwie *housing.csv*, w którym mieszczą się wszystkie interesujące nas dane.

Możesz pobrać to archiwum z poziomu przeglądarki i rozpakować je za pomocą komendy `tar -xzf housing.tgz`, ale lepiej stworzyć w tym celu niewielką funkcję. Funkcja pobierająca dane przydaje się zwłaszcza w sytuacjach, gdy są one regularnie aktualizowane: możesz stworzyć maleńki skrypt uruchamiany za każdym razem, gdy będziesz potrzebować najświeższych danych (ewentualnie możesz zaplanować wykonywanie tej czynności w regularnych odstępach czasu). Automatyzacja tego procesu ułatwia również instalację zbioru danych na wielu komputerach.

Funkcja pobierająca dane przedstawia się następująco[12]:

```
import os
import tarfile
import urllib

DOWNLOAD_ROOT = "https://raw.githubusercontent.com/ageron/handson-ml2/master/"
HOUSING_PATH = os.path.join("zestawy danych", "mieszkania")
HOUSING_URL = DOWNLOAD_ROOT + "zestawy_danych/mieszkania/housing.tgz"

def fetch_housing_data(housing_url=HOUSING_URL, housing_path=HOUSING_PATH):
      os.makedirs(housing_path, exist_ok=True)
      tgz_path = os.path.join(housing_path, "housing.tgz")
      urllib.request.urlretrieve(housing_url, tgz_path)
      housing_tgz = tarfile.open(tgz_path)
      housing_tgz.extractall(path=housing_path)
      housing_tgz.close()
```

Teraz po wywołaniu funkcji `fetch_housing_data()` zostanie utworzony katalog *zestawy danych/mieszkania* w Twojej przestrzeni roboczej, do którego będzie pobrane archiwum *housing.tgz*, po czym zostanie wypakowany z niego plik *housing.csv*.

[11] Musisz również brać pod uwagę ograniczenia prawne, np. prywatne pola, których nie należy nigdy kopiować do niezbyt bezpiecznych magazynów danych.

[12] W prawdziwym projekcie umieścilibyśmy ten kod w pliku Python, na razie jednak wystarczy wstawić go w notatniku Jupyter.

Wczytajmy teraz te dane za pomocą modułu pandas. Także i w tym przypadku stworzymy niewielką funkcję służącą do ich odczytania.

```
import pandas as pd

def load_housing_data(housing_path=HOUSING_PATH):
    csv_path = os.path.join(housing_path, "housing.csv")
    return pd.read_csv(csv_path)
```

Powyższa funkcja zwraca obiekt *DataFrame* modułu pandas przechowujący wszystkie wczytane dane.

Rzut oka na strukturę danych

Przyjrzyjmy się pierwszym pięciu wierszom naszych danych za pomocą metody `head()` (rysunek 2.5)[13].

```
In [5]: housing = load_housing_data()
        housing.head()
```

Out[5]:

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	populatio
0	-122.23	37.88	41.0	880.0	129.0	322.0
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0
2	-122.24	37.85	52.0	1467.0	190.0	496.0
3	-122.25	37.85	52.0	1274.0	235.0	558.0
4	-122.25	37.85	52.0	1627.0	280.0	565.0

Rysunek 2.5. *Pięć pierwszych wierszy w naszym zbiorze danych*

Każdy wiersz reprezentuje jeden dystrykt. Dostępnych jest 10 atrybutów, z czego sześć widzimy na rysunku 2.5: `longitude`, `latitude`, `housing_median_age`, `total_rooms`, `total_bedrooms`, `population`, `households`, `median_income`, `median_house_value` i `ocean_proximity` (w języku polskim są to, odpowiednio: *Dł. geograficzna*, *Szer. geograficzna*, *Mediana wieku mieszkań*, *Całk. liczba pokoi*, *Całk. liczba sypialni*, *Populacja*, *Rodziny*, *Mediana dochodów*, *Mediana cen mieszkań* i *Odległość do oceanu*).

Dzięki metodzie `info()` możemy zapoznać się z krótkim opisem danych, zwłaszcza z całkowitą liczbą wierszy, typem każdego atrybutu oraz liczbą wartości niezerowych (rysunek 2.6).

Na nasz zbiór danych składa się 20 640 przykładów, co oznacza, że jest on dość mały jak na standardy uczenia maszynowego, ale nadaje się idealnie dla początkujących analityków. Zwróć uwagę, że atrybut `total_bedrooms` zawiera zaledwie 20 433 wartości niezerowe, co oznacza, że cecha ta nie została zdefiniowana dla 207 dystryktów. Zajmiemy się tym później.

[13] W celach poglądowych pozostawiam w tym rozdziale nieprzetłumaczone nazwy atrybutów, dzięki czemu po przepisaniu kodu z książki uzyskasz takie same wyniki, jak na rysunkach, natomiast w towarzyszącym notatniku Jupyter znajdziesz wersję tego projektu częściowo przełożoną (tam, gdzie to konieczne) na język polski — *przyp. tłum.*

```
In [6]: housing.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
longitude             20640 non-null float64
latitude              20640 non-null float64
housing_median_age    20640 non-null float64
total_rooms           20640 non-null float64
total_bedrooms        20433 non-null float64
population            20640 non-null float64
households            20640 non-null float64
median_income         20640 non-null float64
median_house_value    20640 non-null float64
ocean_proximity       20640 non-null object
dtypes: float64(9), object(1)
memory usage: 1.6+ MB
```

Rysunek 2.6. *Informacje o zbiorze danych*

Z wyjątkiem pola `ocean_proximity` wszystkie pozostałe atrybuty mają wartości numeryczne. Typem wartości w cesze `ocean_proximity` jest `object`, co oznacza, że można tu przechowywać dowolny obiekt języka Python. Skoro jednak wczytaliśmy dane z pliku CSV, to wiemy, że musi to być wartość tekstowa. Po przyjrzeniu się pierwszym pięciu wierszom zbioru danych okazuje się, że wartości w kolumnie `ocean_proximity` są powtarzalne, dzięki czemu możemy wywnioskować, że mamy najprawdopodobniej do czynienia z atrybutem kategorialnym. Możemy sprawdzić, jakie kategorie są dostępne oraz jaki jest rozkład poszczególnych dystryktów do każdej z nich; posłużymy się w tym celu metodą `value_counts()`:

```
>>> housing["ocean_proximity"].value_counts()
<1H OCEAN 9136
INLAND 6551
NEAR OCEAN 2658
NEAR BAY 2290
ISLAND 5
Name: ocean_proximity, dtype: int64
```

Przyjrzyjmy się innym polom. Metoda `describe()` generuje podsumowanie atrybutów numerycznych (rysunek 2.7).

```
In [8]: housing.describe()
```

Out[8]:

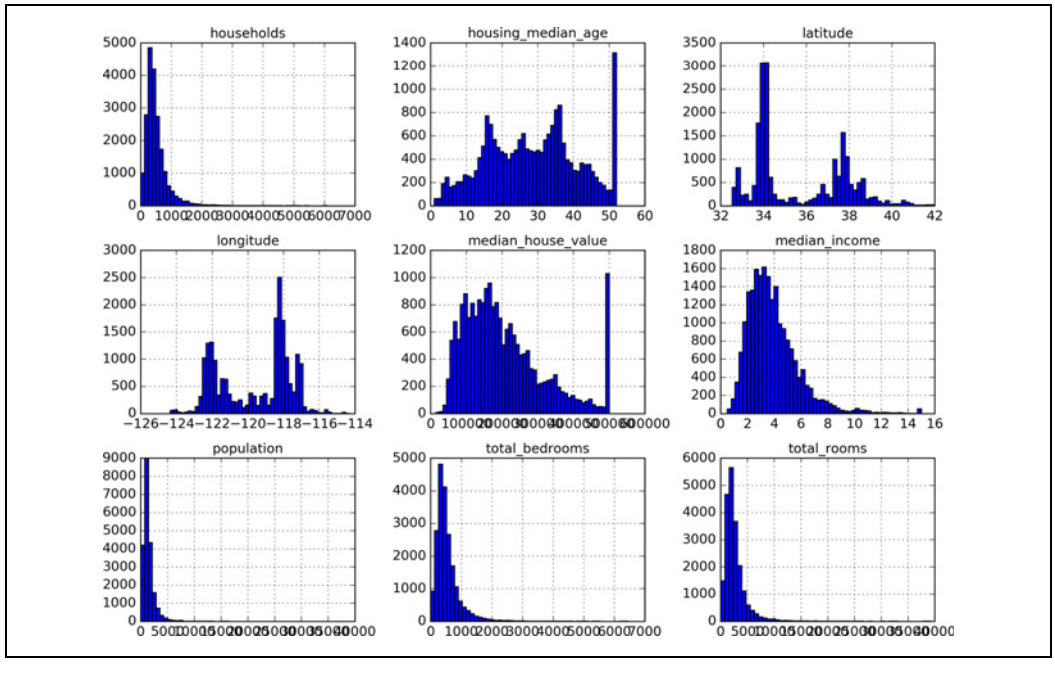
	longitude	latitude	housing_median_age	total_rooms	total_bedrc
count	20640.000000	20640.000000	20640.000000	20640.000000	20433.000(
mean	-119.569704	35.631861	28.639486	2635.763081	537.87055:
std	2.003532	2.135952	12.585558	2181.615252	421.38507(
min	-124.350000	32.540000	1.000000	2.000000	1.000000
25%	-121.800000	33.930000	18.000000	1447.750000	296.00000(
50%	-118.490000	34.260000	29.000000	2127.000000	435.00000(
75%	-118.010000	37.710000	37.000000	3148.000000	647.00000(
max	-114.310000	41.950000	52.000000	39320.000000	6445.0000(

Rysunek 2.7. *Podsumowanie wszystkich atrybutów numerycznych*

Wartości `count` (liczba), `mean` (średnia), `min` i `max` raczej nie wymagają wyjaśnienia. Zwróć uwagę, że wartości zerowe są ignorowane (z tego powodu liczba elementów `total_bedrooms` wynosi tu 20 433, a nie 20 640). Wiersz `std` zawiera **odchylenie standardowe** (ang. *standard deviation*), określające „rozrzut" wartości[14]. Wartości 25%, 50% i 75% ukazują odpowiadające im **percentyle**: percentyl wskazuje wartość, poniżej której znajduje się określony odsetek obserwowanych przykładów. Przykładowo, 25% dystryktów ma wartość atrybutu `housing_median_age` mniejszą od 18, z kolei 50% próbek nie przekracza wartości 29, a 75% procent nie osiągnęło wieku 37 lat. Parametry te są często nazywane, kolejno: 25. percentylem (lub pierwszym **kwartylem**), medianą oraz 75. percentylem (lub trzecim kwartylem).

Kolejnym szybkim sposobem przyjrzenia się analizowanym danym jest wygenerowanie histogramu dla każdego atrybutu numerycznego. Histogram przedstawia liczbę przykładów (w pionowej osi) znajdujących się w określonym przedziale wartości (oś pozioma). Możesz stworzyć histogram jednego atrybutu lub za pomocą metody `hist()` dokonać tego dla całego zbioru danych, co spowoduje narysowanie histogramu dla każdego atrybutu numerycznego (rysunek 2.8 i poniższy listing):

```
%matplotlib inline # Wyłącznie w notatniku Jupyter
import matplotlib.pyplot as plt
housing.hist(bins=50, figsize=(20,15))
plt.show()
```



Rysunek 2.8. *Histogram każdego atrybutu numerycznego*

[14] Odchylenie standardowe jest zwyczajowo oznaczane grecką literą σ (sigma) i w ujęciu matematycznym stanowi pierwiastek kwadratowy z **wariancji**, którą z kolei definiujemy jako średnią arytmetyczną kwadratów odchyleń od wartości średnich przykładów. Gdy wykres danej cechy przyjmuje dzwonowy kształt **rozkładu normalnego** (zwanego również **rozkładem Gaussa**), co jest bardzo często spotykane, znajduje zastosowanie tzw. **reguła 68-95 – 99,7**: mniej więcej 68% próbek znajduje się w odległości 1σ od wartości oczekiwanej, 95% w odległości 2σ, a 99,7% — w odległości 3σ.



Metoda `hist()` korzysta z modułu Matplotlib, który z kolei jest uzależniony od platformy graficznej zdefiniowanej przez użytkownika. Zatem zanim zostanie narysowany jakikolwiek wykres, musisz wybrać platformę używaną przez moduł Matplotlib. Najprostszym rozwiązaniem jest wprowadzenie magicznego polecenia aplikacji Jupyter: `%matplotlib inline`. W ten sposób moduł Matplotlib zostaje skonfigurowany do korzystania z „zaplecza" notatników Jupyter. Oznacza to, że wykresy są generowane w samym notatniku. Zwróć uwagę, że wywołanie funkcji `show()` jest całkowicie dobrowolne w notatnikach Jupyter, gdyż wykres zostanie tak czy inaczej narysowany po uruchomieniu odpowiedniej komórki.

Możemy wyczytać z tych histogramów kilka informacji:

1. Po pierwsze, atrybut mediany dochodów nie przypomina danych podawanych w dolarach amerykańskich (USD). Po konsultacji z zespołem odpowiedzialnym za zebranie tych danych wiesz już, że dane te zostały przeskalowane i ograniczone do maksymalnej wartości 15 (w rzeczywistości 15,0001) dla wyższej mediany dochodów oraz do minimalnej wartości 0,5 (w rzeczywistości 0,4999) dla niższej mediany dochodów. Liczby reprezentują w przybliżeniu dziesiątki tysięcy dolarów (np. wartość 3 oznacza w rzeczywistości ok. 30 000 dolarów). Praca ze wstępnie przetworzonymi danymi stanowi normę w uczeniu maszynowym i niekoniecznie musi stanowić problem, należy jednak spróbować zrozumieć, jakim operacjom zostały one poddane.
2. Ograniczeniu uległy również wartości median wieku oraz cen mieszkań. Ta druga informacja może nas szczególnie zmartwić, ponieważ stanowi ona nasz docelowy atrybut (etykietę). Algorytmy uczenia maszynowego mogą uznać, że ceny domów nigdy nie przekraczają górnej, ograniczonej wartości. Musisz skontaktować się z zespołem klienckim (zespołem wykorzystującym uzyskane przez Ciebie wyniki) i dowiedzieć się, czy ograniczenie to będzie stanowiło problem. Jeżeli powiedzą Ci, że potrzebne są im precyzyjne prognozy nawet powyżej wartości 500 000 dolarów, to pozostają Ci dwie możliwości:
   a. uzyskać prawidłowe etykiety dla dystryktów mających obcięte górne wartości cen;
   b. usunąć te dystrykty z zestawu uczącego (a także testowego, ponieważ system nie powinien być karany, jeżeli będzie przewidywał wartości przekraczające 500 000 dolarów).
3. Każdy z tych atrybutów jest ukazany w odmiennych skalach, nieraz znacznie zróżnicowanych. Zajmiemy się tym zagadnieniem w dalszej części rozdziału, podczas omawiania skalowania cech.
4. Wiele histogramów cechuje się **rozkładem długoogonowym** (ang. *heavy-tailed*): rozciągają się one znacznie bardziej po prawej stronie mediany niż po lewej. Utrudnia to nieco niektórym algorytmom uczenia maszynowego rozpoznawanie wzorców. Spróbujemy później przekształcić te atrybuty w taki sposób, aby ich rozkład był bardziej dzwonowy.

Mam nadzieję, że już wiesz co nieco na temat danych, którymi będziemy się zajmować.



Zaczekaj! Zanim znowu zajrzysz do danych, musisz stworzyć z nich podzbiór testowy, odłożyć go i nigdy więcej do niego nie zerkać.

Stwórz zbiór testowy

Być może dziwnym pomysłem wydaje się dobrowolne odłożenie części danych na tym etapie. Przecież ledwie zajrzeliśmy do nich, aby poznać ich strukturę, i z pewnością moglibyśmy wyciągnąć na ich temat znacznie więcej wniosków przed wybraniem jakiegoś algorytmu, prawda? Owszem, z tym że Twój mózg zawiera niesamowity układ rozpoznawania wzorców, co oznacza, że jest znacznie narażony na przetrenowanie: jeśli przyjrzysz się zbiorowi testowemu, możesz dostrzec jakiś pozornie interesujący wzorzec, który sprawi, że wybierzesz określony model uczenia maszynowego. Po oszacowaniu błędu uogólniania za pomocą tego zbioru uzyskane wyniki okażą się nadmiernie optymistyczne i uruchomisz system, którego wydajność będzie mniejsza od zakładanej. Zjawisko to jest nazywane **obciążeniem związanym z podglądaniem danych** (ang. *data snooping bias*).

Teoretycznie tworzenie zbioru testowego nie powinno być zbyt kłopotliwe: wystarczy wybrać losowo część przykładów (najczęściej 20% całego zbioru lub mniej, jeżeli zestaw danych jest bardzo duży) i je odłożyć.

```
import numpy as np

def split_train_test(data, test_ratio):
    shuffled_indices = np.random.permutation(len(data))
    test_set_size = int(len(data) * test_ratio)
    test_indices = shuffled_indices[:test_set_size]
    train_indices = shuffled_indices[test_set_size:]
    return data.iloc[train_indices], data.iloc[test_indices]
```

Teraz możesz użyć powyższej funkcji w następujący sposób[15]:

```
>>> train_set, test_set = split_train_test(housing, 0.2)
>>> len(train_set)
16512
>>> len(test_set)
4128
```

To rozwiązanie działa, ale nie jest idealne: przy następnym uruchomieniu programu zostanie wygenerowany zupełnie inny zbiór testowy! Po pewnym czasie Ty (albo algorytm uczenia maszynowego) zobaczysz cały zbiór danych uczących, a tego właśnie chcemy uniknąć.

Jednym z rozwiązań jest zapisanie zestawu testowego po jego pierwszym utworzeniu i wczytywanie go przy kolejnych okazjach. Możesz również skorzystać z zarodka liczb losowych (np. `random.seed(42)`)[16] przed wywołaniem funkcji `np.random.permutation()` i uzyskiwać za każdym razem te same pseudolosowe indeksy.

Obydwa te rozwiązania jednak zawiodą po dostarczeniu nowej porcji zaktualizowanych danych. Aby dysponować stabilnym podziałem na przykłady uczące/testowe nawet po zaktualizowaniu zestawu danych, możemy dobierać przykłady stanowiące część zbioru testowego za pomocą ich identyfika-

[15] W niniejszej książce listingi składające się zarówno z kodu, jak i z wyników (tak jak w omawianym przypadku) są dla zwiększenia czytelności formatowane na wzór interpretera Pythona: wiersze kodu mają na początku oznaczenie >>> (lub ... w przypadku wcięcia), natomiast rezultaty nie zawierają żadnego przedrostka.

[16] Ludzie często korzystają z zarodka o wartości 42. Liczba ta nie jest w żaden sposób wyjątkowa, stanowi jedynie odpowiedź na odwieczne pytanie o źródło życia, wszechświata i wszystkiego.

torów (przy założeniu, że próbki te zawierają niepowtarzalne i niezmienne identyfikatory). Możesz na przykład obliczyć hasz każdego identyfikatora i umieścić ten przykład w zestawie danych testowych, jeżeli wartość hasza jest mniejsza lub równa 20% maksymalnej wartości hasza. Dzięki temu jesteśmy w stanie zagwarantować stabilność zestawu testowego za każdym razem, gdy uruchamiamy program, nawet po odświeżeniu zbioru danych. Nowy zbiór testowy będzie zawierał 20% świeżych przykładów, ale nie pojawi się w nim żaden przykład występujący wcześniej w zbiorze uczącym. Oto przykładowa implementacja tego mechanizmu:

```
from zlib import crc32

def test_set_check(identifier, test_ratio):
    return crc32(np.int64(identifier)) & 0xffffffff < test_ratio * 2**32

def split_train_test_by_id(data, test_ratio, id_column):
    ids = data[id_column]
    in_test_set = ids.apply(lambda id_: test_set_check(id_, test_ratio))
    return data.loc[~in_test_set], data.loc[in_test_set]
```

Niestety zbiór danych *Housing* nie zawiera kolumny z identyfikatorami. Najprostszym rozwiązaniem okazuje się wykorzystanie w tym celu indeksu wiersza.

```
housing_with_id = housing.reset_index() # Dodaje kolumnę `index`
train_set, test_set = split_train_test_by_id(housing_with_id, 0.2, "index")
```

Jeśli używasz indeksu wiersza jako niepowtarzalnego identyfikatora, musisz się upewnić, że nowe informacje będą umieszczane na końcu zestawu danych i że żaden wiersz nie zostanie nigdy usunięty. Jeśli nie jesteś w stanie tego zagwarantować, możesz spróbować wykorzystać najstabilniejsze cechy do stworzenia niepowtarzalnego identyfikatora. Przykładowo, współrzędne geograficzne dystryktów raczej nie zmienią się przez kilka najbliższych milionów lat, dlatego możesz wygenerować z nich identyfikatory w zaprezentowany poniżej sposób[17]:

```
housing_with_id["id"] = housing["longitude"] * 1000 + housing["latitude"]
train_set, test_set = split_train_test_by_id(housing_with_id, 0.2, "id")
```

Moduł Scikit-Learn zawiera kilka różnych funkcji rozdzielających zbiory danych na wiele podzbiorów. Najprostszą z nich jest `train_test_split()`, której działanie w znacznym stopniu przypomina zdefiniowaną przed chwilą funkcję `split_train_test()`, zawiera jednak ona pewne dodatkowe elementy. Po pierwsze mamy tu do czynienia z parametrem `random_state`, pozwalającym wybrać zarodek liczb losowych. Po drugie natomiast możemy przekazywać wiele zbiorów danych mających taką samą liczbę wierszy — będą one rozdzielane pomiędzy takie same indeksy (jest to bardzo przydatne rozwiązanie w przypadku, gdy np. etykiety znajdują się w osobnym obiekcie *DataFrame*):

```
from sklearn.model_selection import train_test_split

train_set, test_set = train_test_split(housing, test_size=0.2, random_state=42)
```

[17] W rzeczywistości podane współrzędne geograficzne nie są zbyt precyzyjne i w związku z tym wiele dystryktów otrzyma taką samą wartość identyfikatora oraz znajdzie się w tym samym zbiorze (uczącym bądź testowym). Ku naszemu utrapieniu stanowi to jedno ze źródeł obciążenia próbkowania.

Do tej pory rozważaliśmy metody całkowicie losowego próbkowania. Zazwyczaj takie rozwiązanie spisuje się dobrze w przypadku odpowiednio dużych zbiorów danych (zwłaszcza w odniesieniu do liczby atrybutów), w przeciwnym razie jednak ryzykujemy wprowadzenie dużego obciążenia próbkowania. Gdy urząd statystyczny zamierza przeprowadzić ankietę na tysiącu osób, nie są one wybierane losowo z książki telefonicznej. Muszą zostać tak dobrane, aby stanowiły reprezentatywny przykład całej populacji. Przykładowo, populacja Stanów Zjednoczonych składa się w 51,3% z kobiet i 48,7% z mężczyzn, zatem prawidłowo przeprowadzona ankieta powinna uwzględniać te proporcje: 513 kobiet i 487 mężczyzn. Jest to tzw. **losowanie warstwowe** (ang. *stratified sampling*): populacja zostaje rozdzielona na jednorodne podgrupy zwane **warstwami** (ang. *strata*, l. poj. *stratum*) i z każdej z nich jest dobierana odpowiednia liczba przykładów zapewniająca prawidłowe odzwierciedlenie stanu populacji. Gdyby ankieterzy używali metod czysto losowego doboru przykładów, istniałoby 12-procentowe prawdopodobieństwo uzyskania wypaczonego zestawu testowego zawierającego mniej niż 49% lub ponad 54% kobiet. W każdym przypadku wyniki ankiety cechowałyby się znacznym obciążeniem próbkowania.

Załóżmy, że po rozmowie z ekspertami okazuje się, że bardzo ważnym atrybutem pomagającym w prognozowaniu mediany cen mieszkań jest mediana dochodów. Możesz chcieć zagwarantować, żeby zbiór testowy wiernie reprezentował różne kategorie dochodów dla całego zbioru danych. Mediana dochodów stanowi atrybut ciągłych wartości numerycznych, dlatego najpierw należy stworzyć atrybuty kategorii dochodów. Przyjrzyjmy się uważniej histogramowi mediany dochodów na rysunku 2.8: większość wartości mieści się w przedziale 1,5 do 6 (tzn. 15 000 – 60 000 dolarów), jednak niektóre znacznie przekraczają 6. Ważne jest, aby każda warstwa zestawu danych zawierała wystarczającą liczbę przykładów, gdyż w przeciwnym wypadku oszacowanie znaczenia danej warstwy może być nieadekwatne do rzeczywistości. Oznacza to, że nie możemy tworzyć zbyt wielu warstw i każda z nich powinna być wystarczająco duża. Poniższy kod wykorzystuje funkcję `pd.cut()` do utworzenia atrybutu kategorii dochodów składającego się z pięciu kategorii (oznaczonych cyframi od 1 do 5): zakres pierwszej kategorii wynosi od 0 do 1,5 (tzn. poniżej 15 000 dolarów), drugiej kategorii od 1,5 do 3 itd.:

```
housing["income_cat"] = pd.cut(housing["median_income"],
                               bins=[0., 1.5, 3.0, 4.5, 6., np.inf],
                               labels=[1, 2, 3, 4, 5])
```

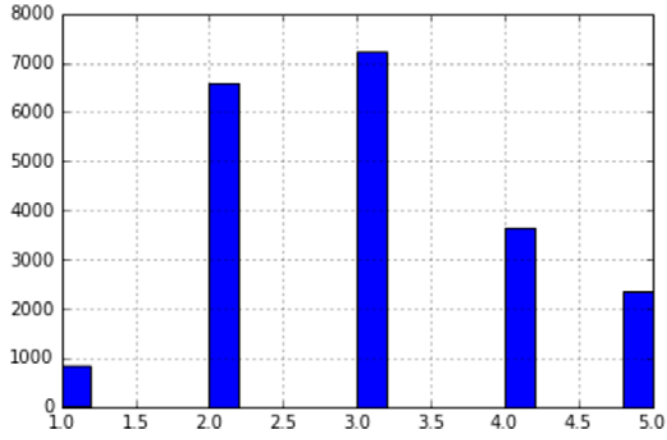
Uzyskane kategorie zostały pokazane na rysunku 2.9:

```
housing["income_cat"].hist()
```

Teraz możemy przeprowadzić próbkowanie warstwowe na podstawie kategorii dochodów. Użyjemy w tym celu klasy `StratifiedShuffleSplit` modułu Scikit-Learn:

```
from sklearn.model_selection import StratifiedShuffleSplit

split = StratifiedShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
for train_index, test_index in split.split(housing, housing["income_cat"]):
   strat_train_set = housing.loc[train_index]
   strat_test_set = housing.loc[test_index]
```



Rysunek 2.9. *Histogram kategorii dochodów*

Sprawdźmy, czy mechanizm ten działa zgodnie z naszymi założeniami. Najpierw możemy zobaczyć proporcje kategorii dochodów w zestawie testowym:

```
>>> strat_test_set["income_cat"].value_counts() / len(strat_test_set)
3     0.350533
2     0.318798
4     0.176357
5     0.114583
1     0.039729
Name: income_cat, dtype: float64
```

Za pomocą podobnego kodu możemy mierzyć proporcje kategorii przychodów w pełnym zestawie danych. Na rysunku 2.10 porównujemy te proporcje w całym zbiorze danych, w zbiorze testowym wygenerowanym za pomocą losowania warstwowego oraz w zestawie testowym utworzonym w całkowicie losowy sposób. Jak widać, zbiór testowy wygenerowany przy użyciu losowania warstwowego ma proporcje niemal identyczne, jak uzyskane w pełnym zbiorze danych, podczas gdy wyniki uzyskane w całkowicie losowym zestawie danych wskazują wypaczenie proporcji.

	L. warstwowe	Losowe	Łącznie	Błąd - losowe (%)	Błąd - l. warstwowe (%)
1.0	0.039729	0.040213	0.039826	0.973236	-0.243309
2.0	0.318798	0.324370	0.318847	1.732260	-0.015195
3.0	0.350533	0.358527	0.350581	2.266446	-0.013820
4.0	0.176357	0.167393	0.176308	-5.056334	0.027480
5.0	0.114583	0.109496	0.114438	-4.318374	0.127011

Rysunek 2.10. *Porównanie obciążenia próbkowania dla losowania warstwowego i losowego próbkowania*

Teraz powinniśmy usunąć atrybut `income_cat`, dzięki czemu dane zostaną przywrócone do pierwotnego stanu:

```
for set_ in (strat_train_set, strat_test_set):
    set_.drop("income_cat", axis=1, inplace=True)
```

Nie bez powodu poświęciliśmy tyle czasu na zagadnienie tworzenia zbioru testowego: jest to często bagatelizowany, ale jeden z najważniejszych elementów uczenia maszynowego. Do tego wiele z poruszonych tu koncepcji przyda nam się w dalszej części książki, podczas omawiania sprawdzianu krzyżowego. Przejdźmy teraz do następnego etapu: eksplorowania danych.

Odkrywaj i wizualizuj dane, aby zdobywać nowe informacje

Na razie zerknęliśmy jedynie na dane w celu ustalenia rodzaju informacji, z jakimi będziemy pracować. Teraz naszym celem jest zagłębienie się w te dane.

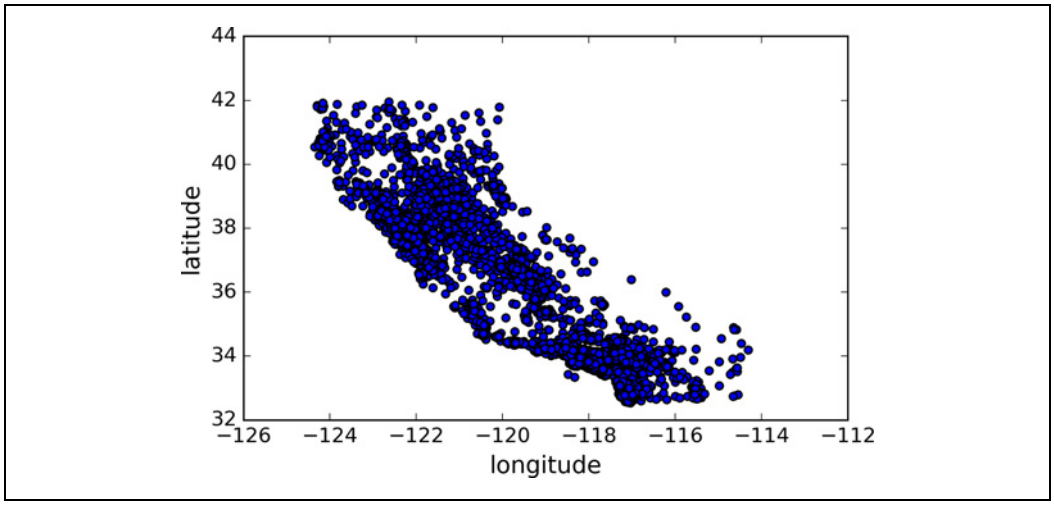
Upewnijmy się najpierw, że odłożyliśmy zbiór testowy i że zajmujemy się wyłącznie zestawem uczącym. Poza tym jeśli zbiór uczący ma bardzo duże rozmiary, warto wydzielić z niego zbiór eksploracyjny, aby przyśpieszyć i ułatwić sobie manipulowanie danymi. W naszym przypadku zestaw uczący jest na tyle mały, że możemy korzystać ze wszystkich danych. Stwórzmy jego kopię, aby móc się nim bawić bez strachu, że go w jakiś sposób uszkodzimy:

```
housing = strat_train_set.copy()
```

Wizualizowanie danych geograficznych

Skoro mamy do czynienia z danymi geograficznymi (długością i szerokością geograficzną), warto stworzyć wykres punktowy wszystkich dystryktów, aby zobaczyć ich rozmieszczenie w układzie współrzędnych (rysunek 2.11):

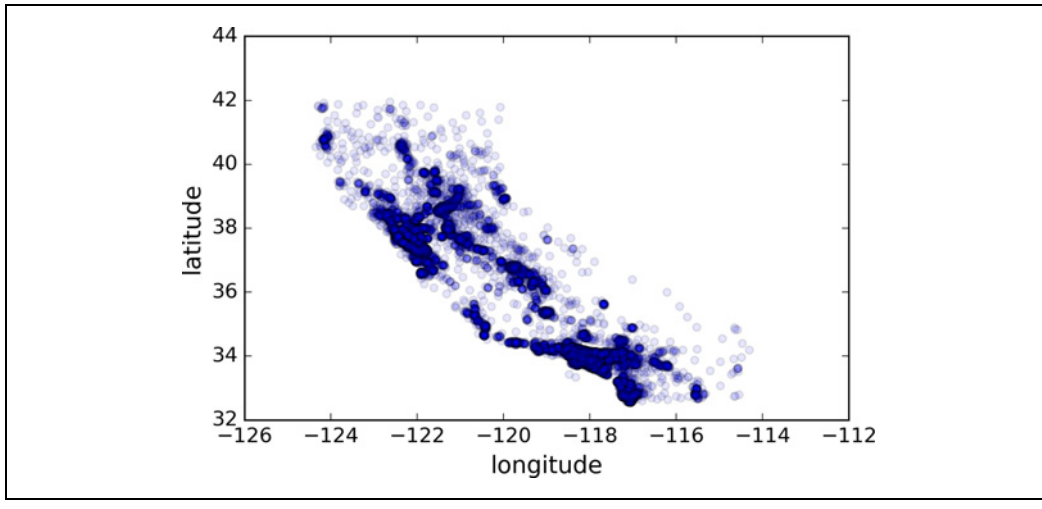
```
housing.plot(kind="scatter", x="longitude", y="latitude")
```



Rysunek 2.11. *Geograficzny wykres punktowy danych*

Kształt tego wykresu rzeczywiście przypomina Kalifornię, jednak poza tym ciężko doszukać się jakiegokolwiek wzoru. Po wyznaczeniu wartości `0,1` parametru `alpha` będzie nam o wiele łatwiej zwizualizować miejsca, w których występuje duże zagęszczenie punktów danych (rysunek 2.12):

```
housing.plot(kind="scatter", x="longitude", y="latitude", alpha=0.1)
```



Rysunek 2.12. *Lepsza wizualizacja, ukazująca obszary o dużym zagęszczeniu punktów danych*

Teraz jest znacznie lepiej: wyraźnie widać obszary o dużym zagęszczeniu dystryktów, mianowicie rejon Bay Area oraz okolice miast Los Angeles i San Diego, a także długi pas w Dolinie Kalifornijskiej, zwłaszcza wokół miast Sacramento i Fresno.

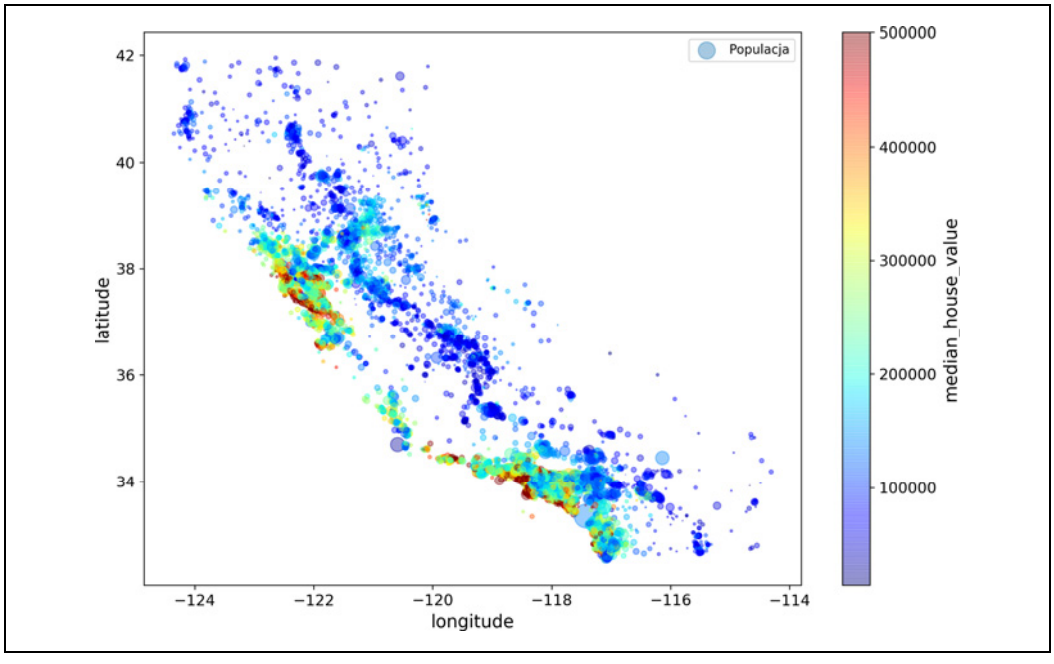
Nasze mózgi doskonale radzą sobie z dostrzeganiem wzorów na danych przedstawionych w postaci graficznej, ale czasami trzeba sobie pomagać, modyfikując niektóre parametry wizualizacji.

Przyjrzyjmy się teraz sytuacji z cenami mieszkań (rysunek 2.13). Promień każdego widocznego kółka symbolizuje populację dystryktu (opcja `s`), z kolei kolorami oznaczamy ceny mieszkań (opcja `c`). Skorzystamy z domyślnej mapy kolorów (opcja `cmap`) o nazwie `jet`, której zakres mieści się od niebieskiego (małe ceny) do czerwonego (duże ceny)[18]:

```
housing.plot(kind="scatter", x="longitude", y="latitude", alpha=0.4,
   s=housing["population"]/100, label="Populacja", figsize=(10,7),
   c="median_house_value", cmap=plt.get_cmap("jet"), colorbar=True,
)
plt.legend()
```

Możemy z tego wykresu wyczytać, że ceny mieszkań są w dużej mierze uzależnione od położenia geograficznego (np. od odległości do oceanu) i zaludnienia, o czym już zapewne wiesz. Algorytm analizy skupień powinien pomóc w wykryciu największego skupiska, a także dodać nowe cechy, pozwalające określić odległość do centrów tego skupienia. Również przydatna może okazać się odległość do oceanu, chociaż w północnej Kalifornii w rejonie wybrzeża ceny domów nie są zbyt wysokie, zatem ta reguła nie jest taka prosta.

[18] Jeżeli czytasz tę książkę w wersji czarno-białej, zaznacz czerwonym pisakiem większość wybrzeża od regionu Bay Area aż do San Diego. Możesz również na żółto otoczyć okolice Sacramento.



Rysunek 2.13. *Ceny mieszkań w Kalifornii: na czerwono zostały zaznaczone wysokie ceny mieszkań, na niebiesko niskie ceny, a duże kółka symbolizują obszary o dużym zaludnieniu*

Poszukiwanie korelacji

Nasz zbiór danych nie jest zbyt duży, dlatego możemy z łatwością wyliczyć **współczynnik korelacji liniowej** (zwany również **współczynnikiem korelacji Pearsona**) pomiędzy każdą parą wartości za pomocą metody `corr()`:

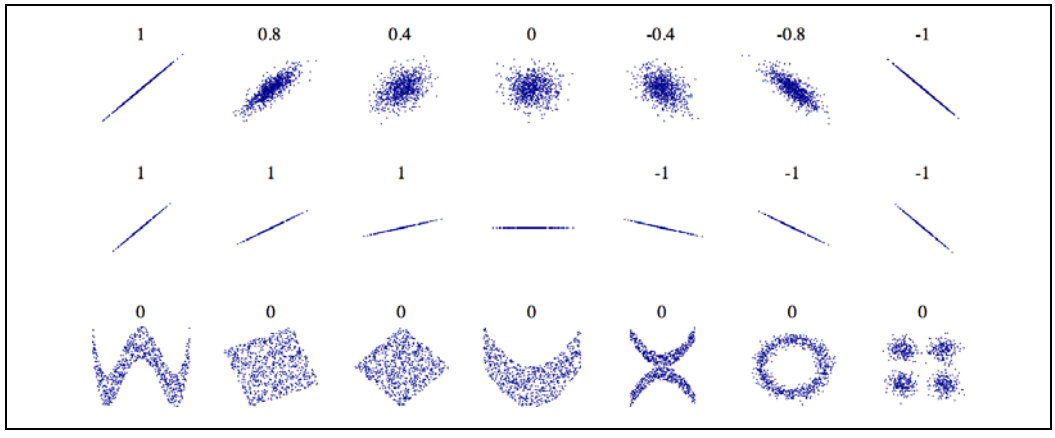
```
corr_matrix = housing.corr()
```

Sprawdźmy teraz stopień korelacji każdego atrybutu z medianą cen mieszkań:

```
>>> corr_matrix["median_house_value"].sort_values(ascending=False)
median_house_value 1.000000
median_income 0.687170
total_rooms 0.135231
housing_median_age 0.114220
households 0.064702
total_bedrooms 0.047865
population -0.026699
longitude -0.047279
latitude -0.142826
Name: median_house_value, dtype: float64
```

Wartości współczynnika korelacji mieszczą się w zakresie pomiędzy –1 a 1. Wartości zbliżone do 1 wskazują silną korelację dodatnią; na przykład mediana cen mieszkań zazwyczaj rośnie wraz ze wzrostem mediany dochodów. Z kolei wartości zbliżone do –1 mówią nam, że istnieje silna korelacja ujemna; widzimy niewielką korelację ujemną pomiędzy szerokością geograficzną a medianą cen mieszkań

(przykładowo ceny nieznacznie spadają, im bardziej kierujemy się na północ). Natomiast wartości bliskie zera oznaczają brak korelacji liniowej. Na rysunku 2.14 widzimy różne wykresy, a także współczynniki korelacji pomiędzy ich osiami odciętych i rzędnych.



Rysunek 2.14. *Współczynnik korelacji liniowej dla różnych zbiorów danych (źródło: Wikipedia; rysunek stanowi własność publiczną)*

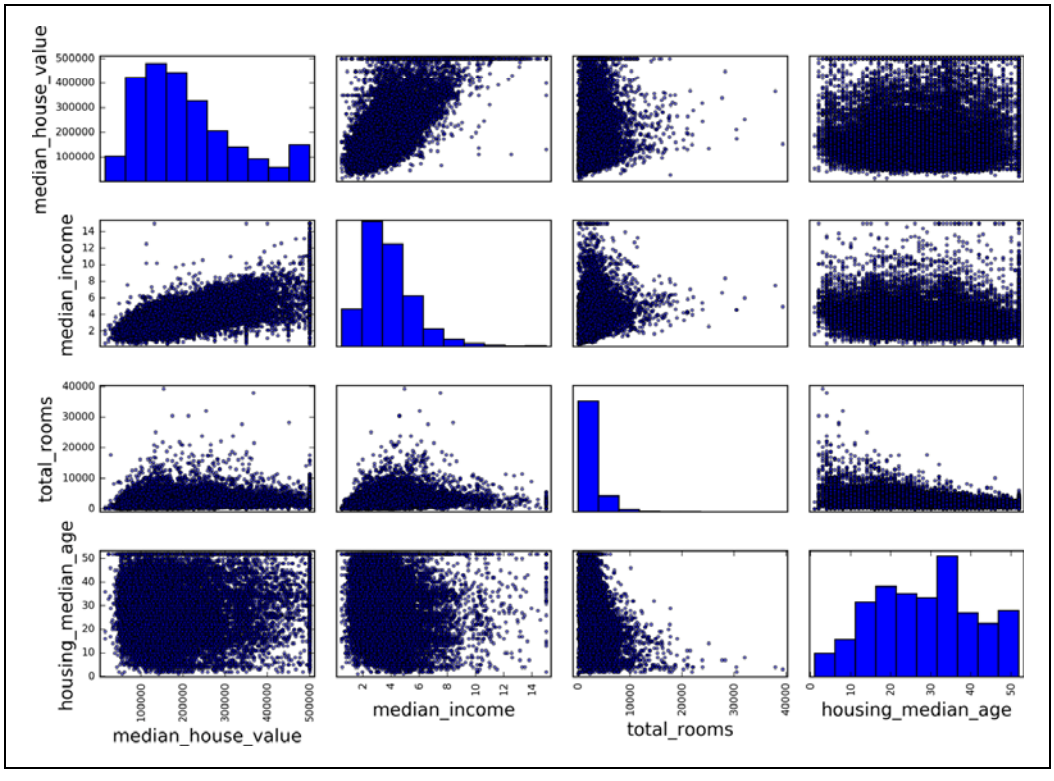


Współczynnik korelacji mierzy jedynie zależność liniową („jeśli wartość x rośnie, to wartość y zazwyczaj rośnie/spada"). Może ona zupełnie nie uwzględniać zależności nieliniowej (np. „jeśli wartość x zbliża się do zera, to wartość y zazwyczaj rośnie"). Zwróć uwagę, że wszystkie wykresy w dolnym rzędzie na rysunku 2.14 mają współczynnik korelacji równy 0 pomimo faktu, że ich osie są w jakiś sposób od siebie zależne: widzimy tu przykład zależności nieliniowej. Ponadto w drugim rzędzie są ukazane przykłady wykresów, w których współczynnik korelacji wynosi 1 lub –1; jak widać, nie ma to żadnego związku z nachyleniem prostej. Przykładowo, Twój wzrost w centymetrach ma współczynnik korelacji liniowej równy 1 dla Twojego wzrostu podanego w metrach lub nanometrach.

Innym sposobem sprawdzenia korelacji pomiędzy atrybutami jest użycie funkcji `scatter_matrix` stanowiącej część modułu pandas; generuje ona wykres każdego atrybutu numerycznego wobec pozostałych atrybutów numerycznych. Obecnie mamy do dyspozycji 11 atrybutów numerycznych, dlatego uzyskalibyśmy w sumie $11^2 = 121$ wykresów, które łącznie nie zmieściłyby się na jednej stronie, dlatego skoncentrujmy się na kilku najbardziej obiecujących atrybutach, które wydają się skorelowane w największym stopniu z medianą cen mieszkań (rysunek 2.15):

```
from pandas.plotting import scatter_matrix

attributes = ["median_house_value", "median_income", "total_rooms",
              "housing_median_age"]
scatter_matrix(housing[attributes], figsize=(12, 8))
```



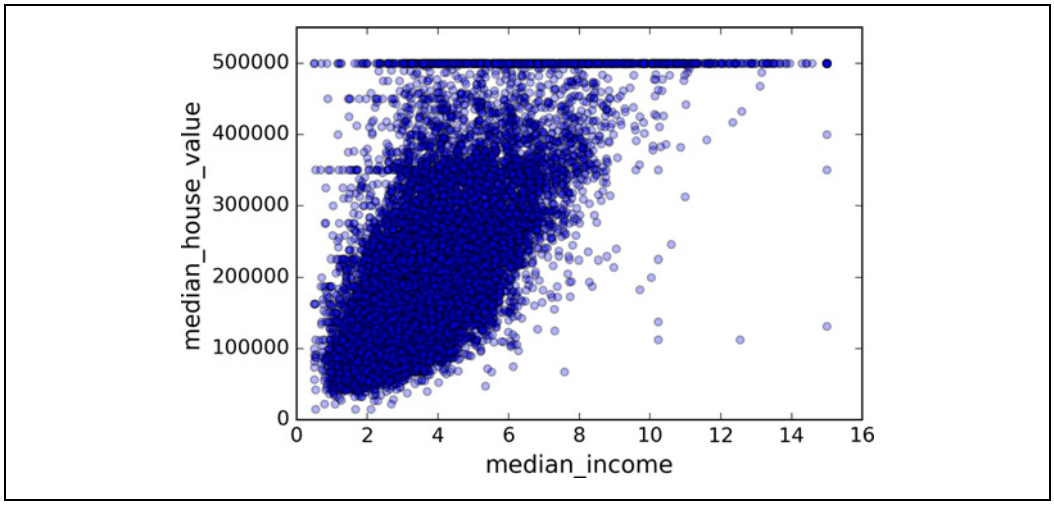
Rysunek 2.15. *Zaprezentowany wykres rozproszenia pokazuje każdy atrybut numeryczny w stosunku do pozostałych atrybutów numerycznych, a także histogram poszczególnych atrybutów*

Główna przekątna (biegnąca od lewego górnego rogu) byłaby wypełniona prostymi, gdyby moduł pandas tworzył wykresy każdej zmiennej wobec samej siebie, co nie byłoby zbyt przydatne. Dlatego zamiast tego widzimy histogram każdego atrybutu (są dostępne również inne opcje; są one dobrze opisane w dokumentacji modułu pandas).

Najbardziej obiecującym atrybutem służącym do prognozowania mediany cen mieszkań jest mediana dochodów, dlatego przyjrzyjmy się uważniej ich wykresowi korelacji (rysunek 2.16):

```
housing.plot(kind="scatter", x="median_income", y="median_house_value", alpha=0.1)
```

Z tego wykresu dowiadujemy się kilku rzeczy. Po pierwsze, korelacja pomiędzy tymi atrybutami jest naprawdę silna; możemy bez trudu dostrzec tendencję wzrostową, a poszczególne punkty znajdują się dość blisko siebie. Po drugie, wspomniane już ograniczenie ceny jest wyraźnie widoczne jako pozioma linia w punkcie 500 000 dolarów. Możemy jednak zauważyć mniej oczywiste proste: poziomą w punkcie 450 000 dolarów, kolejną w okolicy 350 000 dolarów, być może jeszcze jedną mniej więcej w punkcie 280 000 dolarów, a pod nią kilka innych. Warto byłoby spróbować usunąć z danych odpowiedzialne za to dystrykty, aby uniemożliwić algorytmom naukę odtwarzania takich dziwactw.



Rysunek 2.16. *Wykres mediany cen mieszkań w funkcji mediany dochodów*

Eksperymentowanie z kombinacjami atrybutów

Mam nadzieję, że dzięki informacjom zawartym w poprzednich punktach poznałaś/poznałeś już kilka sposobów eksplorowania danych i wydobywania z nich dodatkowej wiedzy. Wykryliśmy kilka artefaktów, które warto byłoby usunąć przed przesłaniem danych do algorytmu uczenia maszynowego, a do tego zauważyliśmy kilka interesujących korelacji pomiędzy atrybutami, zwłaszcza w związku z naszym docelowym atrybutem. Zwróciliśmy także uwagę, że niektóre atrybuty cechują się rozkładem długoogonowym, dlatego wypadałoby je w jakiś sposób przekształcić (np. poprzez wyliczenie ich logarytmu). Oczywiście każdy projekt jest inny, ale ogólne koncepcje są dość podobne.

Ostatnią czynnością, jaką należałoby wykonać przed przygotowaniem danych do użytku algorytmów uczenia maszynowego, jest wypróbowanie różnych kombinacji atrybutów. Przykładowo całkowita liczba pomieszczeń w dystrykcie nie jest zbyt wartościowym atrybutem, jeśli nie znamy liczby przebywających tam rodzin. W rzeczywistości interesuje nas liczba pokojów przypadających na rodzinę. Również całkowita liczba sypialni sama w sobie nic nam nie mówi: prawdopodobnie należałoby ją porównać z całkowitą liczbą pomieszczeń. Inną ciekawą kombinacją atrybutów jest określenie zależności pomiędzy populacją a liczbą rodzin. Stwórzmy teraz wspomniane pary atrybutów:

```
housing["Pokoje_na_rodzinę"] = housing["total_rooms"]/housing["households"]
housing["Sypialnie_na_pokoje"] = housing["total_bedrooms"]/housing["total_rooms"]
housing["Populacja_na_rodzinę"]=housing["population"]/housing["households"]
```

A teraz przyjrzyjmy się uzyskanej macierzy korelacji:

```
>>> corr_matrix = housing.corr()
>>> corr_matrix["median_house_value"].sort_values(ascending=False)
median_house_value 1.000000
median_income 0.687160
Pokoje_na_rodzinę 0.146285
total_rooms 0.135097
housing_median_age 0.114110
households 0.064506
```

```
total_bedrooms 0.047689
Populacja_na_rodzinę -0.021985
population -0.026920
longitude -0.047432
latitude -0.142724
Sypialnie_na_pokoje -0.259984
Name: median_house_value, dtype: float64
```

Całkiem nieźle! Nowy atrybut `Sypialnie_na_pokoje` jest znacznie bardziej skorelowany z medianą cen mieszkań niż całkowita liczba pomieszczeń lub sypialni. Najwidoczniej mieszkania o mniejszym współczynniku liczby sypialni do liczby pomieszczeń okazują się droższe. Liczba pokojów przypadająca na rodzinę również dostarcza nam więcej informacji niż całkowita liczba pomieszczeń w dystrykcie — jest dość oczywiste, że wraz z powierzchnią domu rośnie jego cena.

Ten etap eksploracji danych nie musi być bardzo gruntowny; mamy dzięki niemu dobrze wejść w projekt i szybko uzyskać informacje pomagające w stworzeniu pierwszego odpowiednio dobrego prototypu. Jest to jednak wielokrotnie powtarzany proces: po stworzeniu i uruchomieniu projektu możesz przeanalizować uzyskiwane za jego pomocą wyniki i korzystając ze zdobytych w ten sposób informacji, powrócić do etapu eksploracji danych.

Przygotuj dane pod algorytmy uczenia maszynowego

Czas przygotować dane dla algorytmów uczenia maszynowego. Mamy kilka powodów, aby nie zajmować się tym własnoręcznie, lecz napisać funkcje odpowiedzialne za ten proces:

- W ten sposób będziemy w stanie z łatwością odtwarzać te transformacje na dowolnym zbiorze danych (np. przy okazji całkowicie świeżego zbioru danych).
- Możemy stopniowo powiększać bibliotekę funkcji przekształcających, które możemy wykorzystywać w następnych projektach.
- Możemy używać tych funkcji w już działającym systemie, aby przekształcać nowe dane przed przesłaniem ich do algorytmów uczenia maszynowego.
- Będziemy w stanie z łatwością wypróbowywać różne rodzaje przekształceń i sprawdzać, która kombinacja transformacji sprawuje się najlepiej.

Najpierw jednak powróćmy do pierwotnego zestawu uczącego (poprzez ponowne skopiowanie obiektu `strat_train_set`). Rozdzielmy także czynniki prognostyczne od etykiet, ponieważ nie chcemy dokonywać takich samych przekształceń na obydwu typach danych (zwróć uwagę, że funkcja `drop()` tworzy kopię danych i nie wpływa na zbiór `strat_training_set`):

```
housing = strat_train_set.drop("median_house_value", axis=1)
housing_labels = strat_train_set["median_house_value"].copy()
```

Oczyszczanie danych

Większość algorytmów uczenia maszynowego nie może działać, jeśli brakuje jakichś cech, dlatego stworzymy kilka funkcji zajmujących się tym problemem. Jak wiemy, w atrybucie `total_bedrooms` brakuje kilku wartości, należy więc to zmienić. Mamy trzy możliwości:

1. pozbyć się dystryktów zawierających brakujące dane;
2. pozbyć się całego atrybutu;
3. uzupełnić dane określoną wartością (zero, średnia, mediana itd.).

Możemy tego łatwo dokonać za pomocą metod `dropna()`, `drop()` i `fillna()`, stanowiących część klasy `DataFrame`:

```
housing.dropna(subset=["total_bedrooms"]) # opcja 1
housing.drop("total_bedrooms", axis=1) # opcja 2
median = housing["total_bedrooms"].median() # opcja 3
housing["total_bedrooms"].fillna(median, inplace=True)
```

Jeśli wybierzesz trzecie rozwiązanie, oblicz wartość mediany dla zestawu uczącego i wstaw ją następnie w miejsce brakujących wartości. Nie zapomnij zapisać wartości tejże mediany. Będzie Ci ona później potrzebna do zastąpienia brakujących wartości w zbiorze testowym na etapie oceny działania systemu; posłuży ona również do zastępowania brakujących danych w nowych przykładach.

Moduł Scikit-Learn zawiera przydatną klasę zajmującą się brakującymi wartościami: `SimpleImputer`. Korzystamy z niej w następujący sposób: najpierw tworzymy wystąpienie klasy `SimpleImputer`, w którym zaznaczamy, że chcemy zastąpić brakujące wartości każdego atrybutu medianą tego atrybutu:

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="median")
```

Mediana może być wyliczana jedynie z wartości numerycznych, musisz zatem stworzyć kopię zbioru danych niezawierającą atrybutu `ocean_proximity`:

```
housing_num = housing.drop("ocean_proximity", axis=1)
```

Możemy teraz dopasować wystąpienie klasy `Imputer` do danych uczących za pomocą metody `fit()`:

```
imputer.fit(housing_num)
```

Klasa `imputer` po prostu obliczyła medianę atrybutu i zachowała wyniki w zmiennej `statistics_`. Jedynie w atrybucie `total_bedrooms` brakuje niektórych wartości, nie wiemy jednak, czy po uruchomieniu ostatecznej wersji systemu będą dostępne wszystkie wartości w nowych zestawach danych, dlatego najbezpieczniejszym rozwiązaniem jest zastosowanie klasy `imputer` wobec wszystkich atrybutów numerycznych:

```
>>> imputer.statistics_
array([ -118.51 , 34.26 , 29. , 2119.5 , 433. , 1164. , 408. , 3.5409])
>>> housing_num.median().values
array([ -118.51 , 34.26 , 29. , 2119.5 , 433. , 1164. , 408. , 3.5409])
```

Możemy teraz użyć „wyuczonej” klasy `imputer` do przekształcenia zbioru uczącego poprzez zastąpienie brakujących wartości obliczonymi medianami:

```
X = imputer.transform(housing_num)
```

W wyniku transformacji uzyskujemy klasyczną tablicę NumPy zawierającą przekształcone cechy. Bardzo łatwo jest je umieścić z powrotem w obiekcie `DataFrame`:

```
housing_tr = pd.DataFrame(X, columns=housing_num.columns,
                          index=housing_num.index)
```

Konstrukcja modułu Scikit-Learn

Moduł Scikit-Learn wyróżnia się wyjątkowo dobrze przemyślaną konstrukcją. Oto główne założenia projektowe[19] (*https://arxiv.org/abs/1309.0238*):

- **Jednolitość** — wszystkie obiekty korzystają z jednolitego, prostego interfejsu:
  - **Estymatory** (**funkcje oszacowujące**). Każdy obiekt zdolny do szacowania pewnych parametrów na podstawie zbioru danych jest zwany **estymatorem** (np. klasa `imputer` jest funkcją oszacowującą). Sama operacja szacowania jest wykonywana przez metodę `fit()`, zaś jej jedynym parametrem jest zbiór danych (lub dwa zestawy danych w przypadku algorytmów uczenia nadzorowanego; drugi zbiór zawiera etykiety). Wszelkie inne parametry wpływające na przebieg operacji szacowania są uznawane za hiperparametry (np. parametr `strategy` klasy `imputer`) — muszą być one wyznaczane jako zmienne wystąpienia (generalnie w postaci parametru konstruktora).
  - **Transformatory** (**funkcje transformujące** lub **przekształcające**). Niektóre estymatory (jak na przykład klasa `imputer`) są w stanie również przekształcać zbiór danych; są one zwane **transformatorami**. Także w tym przypadku ich interfejs nie jest skomplikowany: proces transformacji jest przeprowadzany za pomocą metody `transform()`, a jego parametr stanowi modyfikowany zbiór danych. W rezultacie zostaje zwrócony przekształcony zbiór danych. Operacja transformacji zależy przede wszystkim od wyuczonych parametrów (tak jak w przypadku klasy `imputer`). Wszystkie transformatory zawierają również metodę złożoną `fit_transform()`, która jest równoznaczna z wywołaniem metody `fit()`, a po niej `transform()` (czasami jednak ta metoda złożona jest zoptymalizowana i działa znacznie szybciej od jej elementów składowych).
  - **Predyktory** (**funkcje prognostyczne**). Pewne estymatory są w stanie przewidywać wyniki na podstawie zbioru danych; są to tak zwane **predyktory**. Na przykład model `LinearRegression` z poprzedniego rozdziału stanowi predyktor: przewidzieliśmy za jego pomocą satysfakcję z życia, znając wartość PKB per capita danego kraju. Funkcja prognostyczna zawiera metodę `predict()` przyjmującą nowe zbiory danych i zwracającą zestaw powiązanych z nimi prognoz. Dodatkowo występuje tu także metoda `score()`, mierząca jakość prognoz na podstawie zbioru testowego (oraz powiązanych etykiet w przypadku algorytmów uczenia nadzorowanego)[20].
- **Inspekcja** — wszystkie hiperparametry estymatorów są bezpośrednio dostępne poprzez publiczne zmienne wystąpień (np. `imputer.strategy`); wszystkie wyuczone parametry funkcji oszacowującej są dostępne poprzez tego typu zmienne oznaczone na końcu podkreślnikiem (np. `imputer.statistics_`).
- **Nierozprzestrzenianie klas**. Zbiory danych nie są reprezentowane w postaci własnoręcznie przygotowanych klas, lecz jako macierze NumPy lub macierze rzadkie SciPy. Z kolei hiperparametry to standardowe ciągi znaków lub wartości liczbowe języka Python.
- **Kompozycja**. Istniejące elementy składowe są używane tak często, jak to możliwe. Przykładowo, łatwo stworzyć estymator `Pipeline` z samodzielnie dobranej sekwencji funkcji transformujących zakończonych ostatnią funkcją oszacowującą, o czym przekonamy się już niebawem.
- **Rozsądne wartości domyślne**. Moduł Scikit-Learn zawiera przemyślane wartości domyślne dla większości parametrów, dzięki czemu możemy z łatwością tworzyć bazowy system roboczy.

[19] Więcej informacji na temat założeń projektowych znajdziesz w dokumencie *API design for machine learning software: experiences from the scikit-learn project*, Lars Buitinck i in., arXiv preprint arXiv: 1309.0238: 2013.

[20] Niektóre funkcje prognostyczne zawierają również metody mierzące pewność wyliczonych prognoz.

Obsługa tekstu i atrybutów kategorialnych

Do tej pory zajmowaliśmy się wyłącznie atrybutami numerycznymi, przejdźmy jednak teraz do atrybutów tekstowych. W używanym przez nas zestawie danych występuje tylko jeden atrybut tego typu: ocean_proximity. Przyjrzyjmy się wartościom jego pierwszych dziesięciu przykładów:

```
>>> housing_cat = housing[["ocean_proximity"]]
>>> housing_cat.head(10)
              ocean_proximity
17606           <1H OCEAN
18632           <1H OCEAN
14650          NEAR OCEAN
3230                INLAND
3555            <1H OCEAN
19480               INLAND
8879            <1H OCEAN
13685               INLAND
4937            <1H OCEAN
4861            <1H OCEAN
```

Nie jest to typowy tekst: istnieje tu ograniczona liczba wartości, z których każda reprezentuje atrybut kategorialny. Większość algorytmów uczenia maszynowego lepiej sobie radzi z liczbami, przekształćmy więc te kategorie z tekstu na wartości numeryczne. Możemy w tym celu użyć klasy `OrdinalEncoder`[21]:

```
>>> from sklearn.preprocessing import OrdinalEncoder
>>> ordinal_encoder = OrdinalEncoder()
>>> housing_cat_encoded = ordinal_encoder.fit_transform(housing_cat)
>>> housing_cat_encoded[:10]
array([[0.],
       [0.],
       [4.],
       [1.],
       [0.],
       [1.],
       [0.],
       [1.],
       [0.],
       [0.]])
```

Możemy uzyskać listę kategorii za pomocą wystąpienia zmiennej `categories_`. Jest to lista zawierająca jednowymiarową tablicę kategorii dla każdego atrybutu kategorialnego (w tym przypadku mamy do czynienia z listą zawierającą jedną tablicę, ponieważ dysponujemy jednym atrybutem kategorialnym):

```
>>> ordinal_encoder.categories_
[array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR OCEAN'],
       dtype=object)]
```

Istnieje pewien problem z tym rozwiązaniem: algorytmy uczenia maszynowego będą uznawały, że dwie zbliżone wartości będą bardziej do siebie podobne niż do dalszych wartości. W pewnych przypadkach nie stanowi to problemu (np. przy uporządkowanych kategoriach, takich jak „zły”, „przeciętny”, „dobry”, i „znakomity”), ale oczywiście nie dotyczy to kolumny `ocean_proximity`

[21] Klasa ta jest dostępna od wersji 0.20 modułu Scikit-Learn. Jeżeli korzystasz z wcześniejszej wersji, rozważ jego aktualizację lub użycie metody `Series.factorize()` z modułu pandas.

(np. kategorie 0 i 4 są wyraźnie bardziej do siebie podobne niż 0 i 1). Powszechnie stosowanym rozwiązaniem jest stworzenie jednego binarnego atrybutu dla każdej kategorii: jeden atrybut ma wartość 1, gdy kategorią jest `<1H OCEAN` (w przeciwnym wypadku otrzymuje wartość 0), inny atrybut uzyskuje wartość 1 dla kategorii `INLAND` (i 0 dla pozostałych kategorii) itd. Jest to tzw. **kodowanie „gorącojedynkowe”** (ang. *one-hot encoding*), ponieważ tylko jeden atrybut będzie „gorący” (będzie miał wartość 1), podczas gdy pozostałe będą „zimne” (wartość 0). Nowe atrybuty są czasami nazywane **atrybutami sztucznymi** (ang. *dummy attributes*). Moduł Scikit-Learn zawiera koder `OneHotEncoder`, konwertujący kategorialne wartości całkowite na wektory „gorącojedynkowe”[22]:

```
>>> from sklearn.preprocessing import OneHotEncoder
>>> cat_encoder = OneHotEncoder()
>>> housing_cat_1hot = cat.encoder.fit_transform(housing_cat)
>>> housing_cat_1hot
<16512x5 sparse matrix of type '<class 'numpy.float64'>'
with 16512 stored elements in Compressed Sparse Row format>
```

Zauważmy, że w wyniku powyższej operacji uzyskujemy **macierz rzadką** (ang. *sparse matrix*) SciPy, a nie tablicę NumPy. Okazuje się to bardzo użyteczne w przypadku korzystania z atrybutów kategorialnych zawierających tysiące kategorii. Kodowanie „gorącojedynkowe” generuje macierz składającą się z tysięcy kolumn, a cała macierz zawiera tylko jedną wartość 1 na każdy wiersz, pozostałe miejsca są wypełnione zerami. Przechowywanie tak wielu zer w pamięci byłoby marnotrawstwem, dlatego zapisana zostaje nie cała macierz rzadka, lecz położenie poszczególnych elementów niezerowych. Przeważnie możemy z niej korzystać jak ze zwykłej macierzy dwuwymiarowej[23], jeśli jednak naprawdę chcesz ją przekształcić w tablicę NumPy (gęstą), wystarczy wywołać metodę `toarray()`:

```
>>> housing_cat_1hot.toarray()
array([[1., 0., 0., 0., 0.],
       [1., 0., 0., 0., 0.],
       [0., 0., 0., 0., 1.],
       ...,
       [0., 1., 0., 0., 0.],
       [1., 0., 0., 0., 0.],
       [0., 0., 0., 1., 0.]])
```

Możemy również uzyskać listę kategorii za pomocą wystąpienia zmiennej `categories_` kodera:

```
>>> cat_encoder.categories_
[array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR OCEAN'],
       dtype=object)]
```



Jeżeli atrybut kategorialny zawiera znaczną liczbę kategorii (np. kod kraju, zawód, gatunek), to w wyniku kodowania „gorącojedynkowego” będziemy otrzymywać znaczną liczbę cech wejściowych. Może to spowolnić proces uczenia i zmniejszyć skuteczność. W takim przypadku warto zastąpić wejściowe dane kategorialne przydatnymi cechami numerycznymi powiązanymi z kategoriami, np. możesz zastąpić cechę *ocean_proximity* odległością do oceanu (na drodze analogii kod kraju może zostać zastąpiony populacją państwa i PKB per capita). Ewentualnie możesz

[22] Przed wydaniem wersji 0.20 modułu Scikit-Learn metoda ta była w stanie kodować wyłącznie kategorialne wartości całkowite, jednak od wersji 0.20 obsługuje również inne typy danych wejściowych, w tym kategorialne dane tekstowe.

[23] Szczegóły znajdziesz w dokumentacji modułu SciPy.

zamiast każdej kategorii wprowadzić poznawalny, małowymiarowy wektor zwany **wektorem właściwościowym** (ang. *embedding*). Reprezentacja każdej kategorii byłaby poznawana w trakcie uczenia. Jest to przykład **uczenia się reprezentacji** (ang. *representation learning*) (zob. rozdziały 13. i 17.).

Niestandardowe transformatory

Mimo że moduł Scikit-Learn zawiera wiele przydatnych funkcji przekształcających, często jesteśmy zmuszeni pisać własne transformatory wykonujące operacje niestandardowego oczyszczania danych lub łączenia określonych atrybutów. Chcemy, aby nasze funkcje transformujące działały płynnie z funkcjami modułu Scikit-Learn (np. z funkcją potokowania), skoro zaś moduł Scikit-Learn polega na technice inferencji typów (ang. *duck typing*), a nie dziedziczeniu, wystarczy stworzyć klasę i zaimplementować trzy metody: `fit()` (zwracającą wartość obiektu `self`), `transform()` oraz `fit_transform()`.

To ostatnie możemy uzyskać „za darmo", dodając bazową klasę `TransformerMixin`. Jeśli dołączasz klasę bazową `BaseEstimator` (i unikasz zmiennych `*args` oraz `**kargs` w konstruktorze), uzyskujesz także dostęp do dwóch dodatkowych metod (`get_params()` i `set_params()`), które przydadzą się do automatycznego strojenia hiperparametrów. Na przykład, poniżej przedstawiamy malutką klasę transformującą dodającą omówione wcześniej połączone atrybuty:

```
from sklearn.base import BaseEstimator, TransformerMixin

rooms_ix, bedrooms_ix, population_ix, households_ix = 3, 4, 5, 6

class CombinedAttributesAdder(BaseEstimator, TransformerMixin):
   def __init__(self, add_bedrooms_per_room = True): # Żadnych zmiennych *args ani **kargs
      self.add_bedrooms_per_room = add_bedrooms_per_room
   def fit(self, X, y=None):
      return self # Nie robi nic innego
   def transform(self, X):
      Pokoje_na_rodzinę = X[:, rooms_ix] / X[:, households_ix]
      Populacja_na_rodzinę = X[:, population_ix] / X[:, households_ix]
      if self.add_bedrooms_per_room:
         Sypialnie_na_pokoje = X[:, bedrooms_ix] / X[:, rooms_ix]
         return np.c_[X, Pokoje_na_rodzinę, Populacja_na_rodzinę,
            Sypialnie_na_pokoje]
      else:
         return np.c_[X, Pokoje_na_rodzinę, Populacja_na_rodzinę]

attr_adder = CombinedAttributesAdder(add_bedrooms_per_room=False)
housing_extra_attribs = attr_adder.transform(housing.values)
```

W tym przykładzie transformator zawiera jeden hiperparametr, `add_bedrooms_per_room`, którego domyślną wartością jest `True` (warto dawać rozsądne wartości domyślne). Dzięki temu hiperparametrowi z łatwością możesz sprawdzić, czy dodanie atrybutu `Sypialnie_na_pokoje` bardziej pomoże, czy zaszkodzi algorytmom uczenia maszynowego. Mówiąc bardziej ogólnie, za pomocą hiperparametru możesz bramkować każdy etap przygotowywania danych, którego skuteczności nie jesteś pewna/pewny. Im bardziej zautomatyzujesz etapy przygotowywania danych, tym więcej ich kombinacji możesz wypróbować, dzięki czemu znacznie zwiększasz szansę na odkrycie znakomitej kombinacji (i zaoszczędzenie czasu).

Skalowanie cech

Jednym z najważniejszych przekształceń dokonywanych na danych jest **skalowanie cech** (ang. *feature scaling*). Większość algorytmów uczenia maszynowego słabo sobie radzi z atrybutami numerycznymi znajdującymi się w różnych zakresach skali. Dotyczy to również naszego zbioru danych: całkowita liczba pomieszczeń mieści się w zakresie od 6 do 39 320, z kolei wartości mediany dochodów to zakres zaledwie od 0 do 15. Zauważmy, że generalnie nie jest wymagane skalowanie wartości docelowych.

Najczęściej są stosowane dwa rodzaje skalowania wszystkich atrybutów do jednego poziomu: **skalowanie min. – max.** (ang. *min-max scaling*) i **standaryzacja** (ang. *standarization*).

Skalowanie min. – max. (zwane przez wiele osób **normalizacją**) jest najprostszym procesem: wartości są tak skalowane, że mieszczą się w zakresie pomiędzy 0 i 1. Dokonujemy tego, odejmując od danej wartości wartość minimalną i dzieląc otrzymany wynik przez różnicę wartości maksymalnej i minimalnej. W module Scikit-Learn służy do tego funkcja transformująca `MinMaxScaler`. Zawiera ona hiperparametr `feature_range`, pozwalający zmieniać zakres skali, jeśli z jakiegoś powodu nie odpowiada Ci domyślny zakres 0 – 1.

Mechanizm standaryzacji jest odmienny: najpierw odejmujemy od danej wartości średnią (czyli średnia w standaryzowanych próbkach zawsze wynosi 0), a następnie dzielimy ją przez odchylenie standardowe, dzięki czemu wynikowy rozkład ma wariancję jednostkową. W przeciwieństwie do skalowania min. – max. standaryzacja nie ogranicza skalowanych wartości do określonego zakresu, co w przypadku niektórych algorytmów stanowi pewien problem (np. sieci neuronowe często oczekują wartości wejściowych mieszczących się w zakresie 0 – 1). Z drugiej strony, standaryzacja jest znacznie mniej wrażliwa na elementy odstające. Załóżmy, na przykład, że przez pomyłkę mediana dochodów dla danego dystryktu jest równa 100. W przypadku skalowania min. – max. wszystkie pozostałe wartości z przedziału 0 – 15 zostałyby umieszczone w zakresie 0 – 0,15, natomiast nie miałoby to tak wielkiego wpływu na standaryzowane wyniki. Standaryzację w module Scikit-Learn uzyskujemy za pomocą transformatora `StandardScaler`.



Podobnie jak w przypadku wszystkich transformacji, należy dostosować funkcje skalujące wyłącznie do danych uczących, nie do całego zbioru danych (włącznie ze zbiorem testowym). Dopiero po wyuczeniu wobec danych uczących możesz użyć tych funkcji do przekształcania danych testowych (oraz nowych przykładów).

Potoki transformujące

Jak widać, należy przeprowadzać wiele operacji przekształcania we właściwej kolejności. Na szczęście możemy skorzystać z klasy `Pipeline`, pomagającej wyznaczyć odpowiednią sekwencję transformacji. Poniżej przedstawiamy niewielki potok stosowany wobec atrybutów numerycznych:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

num_pipeline = Pipeline([
      ('imputer', SimpleImputer(strategy="median")),
      ('attribs_adder', CombinedAttributesAdder()),
      ('std_scaler', StandardScaler()),
   ])

housing_num_tr = num_pipeline.fit_transform(housing_num)
```

Konstruktor `Pipeline` przyjmuje listę par nazwa/estymator definiującą sekwencję operacji. Wszystkie elementy oprócz ostatniego estymatora muszą być funkcjami przekształcającymi (tj. muszą zawierać metodę `fit_transform()`). Nazwy mogą być dowolne (pod warunkiem że są niepowtarzalne i nie zawierają podwójnego podkreślnika: __); przydadzą się one później podczas strojenia hiperparametrów.

W momencie wywołania metody `fit()` następuje sekwencyjne wywołanie metody `fit_transform()` wobec wszystkich transformatorów, przekazanie wyniku każdego wywołania w postaci parametru do następnego wywołania aż do osiągnięcia ostatniego estymatora, dla którego zostaje wywołana metoda `fit()`.

Potok odsłania te same metody, co ostatni estymator. W tym przykładzie ostatnią funkcją prognostyczną jest `StandardScaler`, będąca w istocie transformatorem, dlatego potok zawiera metodę `transform()`, wykonującą wszystkie przekształcenia wobec sekwencji danych (a także oczywiście metodę `fit_transform()`, z której skorzystaliśmy).

Do tej pory zajmowaliśmy się oddzielnie kolumnami kategorialnymi i numerycznymi. Byłoby wygodniej, gdybyśmy dysponowali pojedynczym transformatorem przetwarzającym wszystkie kolumny i dobierający odpowiednie transformacje do poszczególnych typów kolumn. W tym celu zaprezentowano w wersji 0.20 modułu Scikit-Learn klasę `ColumnTransformer`, a dodatkowa znakomita informacja jest taka, że współpracuje ona z obiektami `DataFrame`. Wprowadźmy za jej pomocą wszystkie przekształcenia zestawu danych *Housing*:

```
from sklearn.compose import ColumnTransformer

num_attribs = list(housing_num)
cat_attribs = ["ocean_proximity"]

full_pipeline = ColumnTransformer([
        ("num", num_pipeline, num_attribs),
        ("cat", OneHotEncoder(), cat_attribs),
    ])

housing_prepared = full_pipeline.fit_transform(housing)
```

Najpierw importujemy klasę `ColumnTransformer`, następnie uzyskujemy listy nazw kolumn numerycznych i nazw kolumn kategorialnych, po czym konstruujemy obiekt `ColumnTransformer`. Konstruktor wymaga listy krotek, gdzie każda krotka zawiera nazwę[24], transformator, a także listę nazw (lub indeksów) kolumn, wobec których powinien zostać użyty transformator. W tym przykładzie decydujemy, że kolumny numeryczne powinny być przekształcane za pomocą uprzednio zdefiniowanego obiektu `num_pipeline`, natomiast do transformacji kolumn kategorialnych wyznaczamy `OneHotEncoder`. Na koniec stosujemy tę klasę `ColumnTransformer` na zestawie danych *Housing*: każdy transformator zostaje użyty wobec odpowiednich kolumn, a wyniki wzdłuż drugiej osi są łączone (transformatory muszą zwracać tę samą liczbę rzędów).

Zwróć uwagę, że obiekt `OneHotEncoder` zwraca macierz rzadką, natomiast `num_pipeline` gęstą. W przypadku występowania takiej mieszanki macierzy rzadkich i gęstych klasa `ColumnTransformer` oszacowuje gęstość macierzy końcowej (tzn. współczynnik występowania komórek niezerowych) i zwraca

[24] Podobnie jak w przypadku potoków nazwa może być dowolna, pod warunkiem że nie zawiera podwójnych podkreślników.

macierz rzadką, jeżeli gęstość jest mniejsza od ustalonego progu (domyślnie `sparse_threshold=0.3`). W tym przykładzie zostaje zwrócona macierz gęsta. I to tyle! Wykorzystujemy potok przetwarzania wstępnego przyjmujący pełny zestawy danych *Housing* i stosujący wobec każdej kolumny odpowiednie przekształcenia.



Zamiast korzystać z transformatora możesz wyznaczyć łańcuch znaków `"drop"`, jeżeli chcesz, aby kolumny zostały usunięte, lub `"passthrough"`, jeśli mają pozostać niezmienione. Domyślnie pozostałe kolumny (te, które nie zostały wymienione w liście) zostaną usunięte, ale możesz wyznaczyć parametr `remainder` w dowolnym transformatorze (lub łańcuchu znaków `"passthrough"`), jeśli chcesz, aby były one przetwarzane inaczej.

Jeżeli korzystasz z wersji 0.19 modułu Scikit-Learn lub wcześniejszej, możesz użyć innej biblioteki, np. `sklearn-pandas`, lub przygotować własny, niestandardowy transformator mający taką samą funkcjonalność jak `ColumnTransformer`. Ewentualnie masz również do dyspozycji klasę `FeatureUnion`, która może stosować odmienne transformatory i łączyć ich wyniki. Nie możesz jednak wyznaczać różnych kolumn dla każdego transformatora — transformatory są stosowane wobec wszystkich danych. Istnieje możliwość obejścia tego ograniczenia poprzez wykorzystanie niestandardowego transformatora przeznaczonego do doboru kolumn (przykład znajdziesz w notatniku Jupyter).

Wybór i uczenie modelu

Nareszcie! Określiliśmy ramy problemu, zdobyliśmy i przeanalizowaliśmy dane, przetestowaliśmy zestawy uczący i testowy, a także napisaliśmy potoki służące do automatycznego oczyszczania i przygotowywania danych pod algorytmy uczenia maszynowego. Jesteśmy gotowi, aby wybrać i wytrenować model uczenia maszynowego.

Trenowanie i ocena modelu za pomocą zbioru uczącego

Mam dobre wieści do przekazania: dzięki wszystkim wcześniejszym czynnościom dalsze etapy będą znacznie prostsze, niż Ci się wydaje. Wyuczmy najpierw model regresji liniowej, podobnie jak dokonaliśmy tego w poprzednim rozdziale.

```
from sklearn.linear_model import LinearRegression

lin_reg = LinearRegression()
lin_reg.fit(housing_prepared, housing_labels)
```

I już! Masz teraz do dyspozycji działający model regresji liniowej. Sprawdźmy kilka przykładów ze zbioru uczącego:

```
>>> some_data = housing.iloc[:5]
>>> some_labels = housing_labels.iloc[:5]
>>> some_data_prepared = full_pipeline.transform(some_data)
>>> print("Prognozy:", lin_reg.predict(some_data_prepared))
Prognozy: [ 210644.6045 317768.8069 210956.4333 59218.9888 189747.5584]
>>> print("Etykiety:", list(some_labels))
Etykiety: [286600.0, 340600.0, 196900.0, 46300.0, 254500.0]
```

Model działa, chociaż prognozy nie są zbyt dokładne (np. pierwsza prognoza odstaje od etykiety niemal o 40%!). Zmierzmy błąd RMSE modelu regresji dla całego zbioru uczącego za pomocą funkcji `mean_squared_error()`:

```
>>> from sklearn.metrics import mean_squared_error
>>> housing_predictions = lin_reg.predict(housing_prepared)
>>> lin_mse = mean_squared_error(housing_labels, housing_predictions)
>>> lin_rmse = np.sqrt(lin_mse)
>>> lin_rmse
68628.19819848922
```

Lepsze to niż nic, ale wynik ten nie jest powalający: wartości atrybutu `Mediana cen mieszkań` dla większości dystryktów mieszczą się w zakresie pomiędzy 120 000 a 265 000 dolarów, dlatego standardowy błąd predykcji rzędu 68 628 dolarów nie jest zbyt satysfakcjonujący. Widzimy tu klasyczny przykład niedotrenowania modelu wobec danych uczących. Taka sytuacja oznacza, że cechy nie dostarczają odpowiedniej ilości informacji pozwalających na uzyskanie dobrych prognoz albo że model nie jest wystarczająco dobry. Jak wiemy z poprzedniego rozdziału, podstawowymi sposobami radzenia sobie z problemem niedotrenowania są wybór potężniejszego algorytmu, wprowadzenie lepszych cech lub zmniejszenie ograniczeń modelu. Nasz model nie jest regularyzowany, dlatego odpada ostatnia możliwość. Możemy spróbować dodać więcej cech (np. logarytm z populacji), najpierw jednak sprawdźmy bardziej skomplikowany model i zobaczmy, jak sobie poradzi.

Wytrenujmy model `DecisionTreeRegressor`. Jest to zaawansowany model potrafiący wyszukiwać w danych skomplikowane, nieliniowe zależności (w rozdziale 6. zajmiemy się szczegółowo tematem drzew decyzyjnych). Użyty kod powinien wyglądać już znajomo:

```
from sklearn.tree import DecisionTreeRegressor

tree_reg = DecisionTreeRegressor()
tree_reg.fit(housing_prepared, housing_labels)
```

Po wyuczeniu modelu sprawdźmy jego wydajność wobec zbioru uczącego:

```
>>> housing_predictions = tree_reg.predict(housing_prepared)
>>> tree_mse = mean_squared_error(housing_labels, housing_predictions)
>>> tree_rmse = np.sqrt(tree_mse)
>>> tree_rmse
0.0
```

Że jak? Żadnego błędu? Czyżby ten model był stuprocentowo bezbłędny? Oczywiście jest o wiele bardziej prawdopodobne, że model po prostu został znacznie przetrenowany. Skąd mamy mieć pewność? Wiemy już, że nie chcemy używać zbioru uczącego, dopóki nie będziemy w pełni przekonani o skuteczności wybranego modelu, dlatego musimy wykorzystać część zbioru uczącego do trenowania, a część do ocenienia modelu.

Dokładniejsze ocenianie za pomocą sprawdzianu krzyżowego

Jednym ze sposobów oceniania modelu drzewa decyzyjnego byłoby wykorzystanie funkcji `train_test_split()` do rozdzielenia zestawu uczącego na podzbiory trenujący i walidacyjny, następnie wyuczenie naszego modelu przy użyciu tego zbioru trenującego, po czym ewaluacja za pomocą zbioru walidacyjnego. Proces ten oznacza dodatkową pracę, ale nie jest zbyt skomplikowany i całkiem dobrze spełnia swoje zadanie.

Doskonałą alternatywą jest użycie funkcji k-krotnego **sprawdzianu krzyżowego** (**kroswalidacji,** ang. *k-fold cross-validation*). Za pomocą poniższego kodu zbiór uczący zostaje losowo rozdzielony na 10 oddzielnych **podzbiorów** (ang. *folds*), następnie przeprowadzane jest dziesięciokrotne trenowanie i ocenianie modelu drzewa decyzyjnego (za każdym razem zostaje wybrany inny podzbiór do oceny wydajności modelu, a dziewięć pozostałych służy do uczenia). W konsekwencji uzyskujemy tablicę zawierającą 10 wyników ewaluacji:

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(tree_reg, housing_prepared, housing_labels,
                         scoring="neg_mean_squared_error", cv=10)
tree_rmse_scores = np.sqrt(-scores)
```



Funkcja sprawdzianu krzyżowego będąca częścią modułu Scikit-Learn oczekuje funkcji użyteczności (im większa wartość, tym lepiej), a nie funkcji kosztu (im mniejsza wartość, tym lepiej), dlatego funkcja zliczająca wynik stanowi w rzeczywistości przeciwieństwo błędu MSE (np. ujemną wartość), dlatego w powyższym kodzie obliczamy wartość –scores przed określeniem pierwiastka kwadratowego.

Spójrzmy na wyniki:

```
>>> def display_scores(scores):
...     print("Wyniki:", scores)
...     print("Średnia:", scores.mean())
...     print("Odchylenie standardowe:", scores.std())
...
>>> display_scores(tree_rmse_scores)
Wyniki:  [70194.33680785 66855.16363941 72432.58244769 70758.73896782
    71115.88230639 75585.14172901 70262.86139133 70273.6325285
    75366.87952553 71231.65726027]
Średnia 71407.68766037929
Odchylenie standardowe: 2439.4345041191004
```

Model drzewa decyzyjnego nie wygląda teraz już tak dobrze. W rzeczywistości wygląda na to, że sprawuje się nawet gorzej niż model regresji liniowej! Zwróć uwagę, że za pomocą sprawdzianu krzyżowego możemy nie tylko oszacować wydajność naszego modelu, lecz także zmierzyć precyzję oszacowań (tj. odchylenie standardowe). Wynik uzyskany dla modelu drzewa decyzyjnego wynosi w przybliżeniu 71 407, plus minus 2439. Nie uzyskalibyśmy takich informacji, gdybyśmy skorzystali wyłącznie z jednego zbioru walidacyjnego. Jednak ceną kroswalidacji jest konieczność kilkukrotnego uczenia modelu, dlatego rozwiązanie to nie zawsze jest dostępne.

Dla pewności przeprowadźmy sprawdzian krzyżowy dla modelu regresji liniowej:

```
>>> lin_scores = cross_val_score(lin_reg, housing_prepared, housing_labels,
...                              scoring="neg_mean_squared_error", cv=10)
...
>>> lin_rmse_scores = np.sqrt(-lin_scores)
>>> display_scores(lin_rmse_scores)
Wyniki: [66782.73843989 66960.118071 70347.95244419 74739.57052552
    68031.13388938 71193.84183426 64969.63056405 68281.61137997
    71552.91566558 67665.10082067]
Średnia: 69052.46136345083
Odchylenie standardowe: 2731.674001798348
```

Zgadza się: model drzewa decyzyjnego ulega tak silnemu przetrenowaniu, że osiąga gorsze wyniki od modelu regresji liniowej.

Sprawdźmy jeszcze jeden model: `RandomForestRegressor`. Jak się przekonamy w rozdziale 7., mechanizm działania modeli losowego lasu polega na uczeniu wielu drzew decyzyjnych za pomocą różnych podzbiorów cech, po czym następuje uśrednienie otrzymanych prognoz. Konstruowanie modelu na podwalinach wielu innych modeli nazywamy metodami **uczenia zespołowego** (ang. *ensemble learning*) — nieraz dzięki temu rozwiązaniu uzyskujemy jeszcze lepszą wydajność stosowanych algorytmów. Pominiemy większą część kodu, ponieważ w większości wygląda on tak samo, jak w przypadku poprzednich modeli:

```
>>> from sklearn.ensemble import RandomForestRegressor
>>> forest_reg = RandomForestRegressor()
>>> forest_reg.fit(housing_prepared, housing_labels)
>>> [...]
>>> forest_rmse
18603.515021376355
>>> display_scores(forest_rmse_scores)
Wyniki: [49519.80364233 47461.9115823 50029.02762854 52325.28068953
    49308.39426421 53446.37892622 48634.8036574 47585.73832311
    53490.10699751 50021.5852922 ]
Średnia: 50182.303100336096
Odchylenie standardowe: 2097.0810550985693
```

O proszę, teraz jest znacznie lepiej: model losowego lasu prezentuje się bardzo obiecująco. Zauważmy jednak, że wynik dla zestawu uczącego jest ciągle mniejszy niż dla zbiorów walidacyjnych, co oznacza, że model ten w dalszym ciągu ulega przetrenowaniu. Mamy do dyspozycji następujące rozwiązania: uproszczenie modelu, jego ograniczenie (np. regularyzacja) lub pozyskanie znacznie większej liczby przykładów uczących. Zanim jednak wybierzemy model losowego lasu, powinniśmy wypróbować wiele innych modeli, reprezentujących różne rodzaje algorytmów uczenia maszynowego (np. kilka maszyn wektorów nośnych przy użyciu różnych jąder i być może sieć neuronową), bez spędzania dużej ilości czasu na strojeniu hiperparametrów. Naszym celem jest stworzenie krótkiej listy (od dwóch do pięciu pozycji) najbardziej obiecujących modeli.



Zapisuj każdy model, z którym eksperymentujesz, dzięki czemu będziesz w stanie szybko powracać do dowolnego modelu. Nie zapomnij zachowywać zarówno wartości hiperparametrów i wyuczonych parametrów, jak i wyników sprawdzianu krzyżowego, a być może nawet samych prognoz. W ten sposób możesz z łatwością porównywać wyniki poszczególnych rodzajów modeli i zwracane przez nie błędy. Możemy zapisywać modele stworzone w module Scikit-Learn za pomocą modułu `pickle` lub `joblib` — ten drugi jest skuteczniejszy w serializowaniu dużych macierzy NumPy (możesz zainstalować tę bibliotekę za pomocą modułu pip):

```
import joblib

joblib.dump(my_model, "mój_model.pkl")
# a później...
my_model_loaded = joblib.load("mój_model.pkl")
```

Wyreguluj swój model

Załóżmy, że masz już sporządzoną listę obiecujących modeli. Musisz je teraz dostroić. Możemy tego dokonać na kilka sposobów.

Metoda przeszukiwania siatki

Jednym z rozwiązań jest własnoręczne dobieranie wartości hiperparametrów, dopóki nie uzyskasz ich znakomitej kombinacji. Jest to bardzo żmudne zajęcie i być może nie masz tyle czasu, aby sprawdzić wszystkie możliwości.

Zamiast tego możemy zlecić poszukiwania obiektowi `GridSearchCV`. Wystarczy podać interesujące nas hiperparametry oraz ich proponowane wartości, a wszystkie kombinacje zostaną ocenione za pomocą sprawdzianu krzyżowego. Na przykład poniższy kod poszukuje najlepszej kombinacji wartości hiperparametrów dla modelu `RandomForestRegressor`:

```
from sklearn.model_selection import GridSearchCV

param_grid = [
   {'n_estimators': [3, 10, 30], 'max_features': [2, 4, 6, 8]},
   {'bootstrap': [False], 'n_estimators': [3, 10], 'max_features': [2, 3, 4]},
]

forest_reg = RandomForestRegressor()

grid_search = GridSearchCV(forest_reg, param_grid, cv=5,
                           scoring='neg_mean_squared_error'
                           return_train_score=True)

grid_search.fit(housing_prepared, housing_labels)
```



Jeśli nie wiesz, jaką wartość wyznaczyć danemu hiperparametrowi, najprostszym rozwiązaniem jest wstawienie kolejnych wielokrotności liczby 10 (lub mniejszych wartości w przypadku bardziej szczegółowego przeszukiwania, co zostało zaprezentowane na przykładzie hiperparametru `n_estimators`).

Parametr `param_grid` najpierw oszacowuje wszystkie 3×4 = 12 kombinacji wartości hiperparametrów `n_estimators` i `max_features` zdefiniowanych w pierwszym obiekcie `dict` (na razie nie jest ważne, do czego służą te hiperparametry; wrócimy do nich w rozdziale 7.), a następnie 2×3 = 6 kombinacji wartości parametrów z drugiego obiektu `dict`, w tym przypadku jednak wyłączamy hiperparametr `bootstrap` (wartość `False`; jego domyślną wartością jest `True`).

Mechanizm przeszukiwania siatki sprawdzi 12+6 = 18 kombinacji wartości hiperparametrów modelu `RandomForestRegressor`, a każdy model zostanie pięciokrotnie wyuczony (gdyż korzystamy z pięciokrotnego sprawdzianu krzyżowego). Innymi słowy, ostatecznie zostanie przeprowadzonych 18×5 = 90 przebiegów uczenia! Może to zająć trochę czasu, ale ostatecznie możesz uzyskać najlepszą kombinację hiperparametrów, np. taką:

```
>>> grid_search.best_params_
{'max_features': 8, 'n_estimators': 30}
```



Wartości 8 i 30 są największe spośród tych, które zostały przez nas wstawione do kodu, dlatego prawdopodobnie warto byłoby przeprowadzić kolejne przeszukiwanie siatki, tym razem z podanymi większymi wartościami — możemy dzięki temu uzyskać jeszcze lepsze wyniki.

Możemy również bezpośrednio uzyskać najlepszy estymator:

```
>>> grid_search.best_estimator_
RandomForestRegressor(bootstrap=True, criterion='mse', max_depth=None,
   max_features=8, max_leaf_nodes=None, min_impurity_decrease=0.0,
   min_impurity_split=None, min_samples_leaf=1, min_samples_split=2,
   min_weight_fraction_leaf=0.0, n_estimators=30, n_jobs=None,
   oob_score=False, random_state=None, verbose=0, warm_start=False)
```



Jeżeli obiekt `GridSearchCV` zostanie zainicjowany z parametrem `refit=True` (jest to wartość domyślna), to po znalezieniu najlepszego estymatora za pomocą sprawdzianu krzyżowego ta funkcja oszacowująca będzie wykorzystana wobec całego zbioru uczącego. Zazwyczaj jest to dobre rozwiązanie, ponieważ im więcej danych zostanie użytych, tym bardziej wzrośnie wydajność modelu.

Oczywiście są również dostępne wyniki ewaluacji:

```
>>> cvres = grid_search.cv_results_
>>> for mean_score, params in zip(cvres["mean_test_score"], cvres["params"]):
...     print(np.sqrt(-mean_score), params)
...
63669.05791727153 {'max_features': 2, 'n_estimators': 3}
55627.16171305252 {'max_features': 2, 'n_estimators': 10}
53384.57867637289 {'max_features': 2, 'n_estimators': 30}
60965.99185930139 {'max_features': 4, 'n_estimators': 3}
52740.98248528835 {'max_features': 4, 'n_estimators': 10}
50377.344409590376 {'max_features': 4, 'n_estimators': 30}
58663.84733372485 {'max_features': 6, 'n_estimators': 3}
52006.15355973719 {'max_features': 6, 'n_estimators': 10}
50146.465964159885 {'max_features': 6, 'n_estimators': 30}
57869.25504027614 {'max_features': 8, 'n_estimators': 3}
51711.09443660957 {'max_features': 8, 'n_estimators': 10}
49682.25345942335 {'max_features': 8, 'n_estimators': 30}
62895.088889905004 {'bootstrap': False, 'max_features': 2, 'n_estimators': 3}
54658.14484390074 {'bootstrap': False, 'max_features': 2, 'n_estimators': 10}
59470.399594730654 {'bootstrap': False, 'max_features': 3, 'n_estimators': 3}
52725.01091081235 {'bootstrap': False, 'max_features': 3, 'n_estimators': 10}
57490.612956065226 {'bootstrap': False, 'max_features': 4, 'n_estimators': 3}
51009.51445842374 {'bootstrap': False, 'max_features': 4, 'n_estimators': 10}
```



Pamiętaj, że możesz traktować niektóre etapy przygotowywania danych jako hiperparametry. Przykładowo, mechanizm przeszukiwania siatki automatycznie sprawdzi, czy należy dodać cechy, co do których nie masz pewności (np. hiperparametr `add_bedrooms_per_room` wyliczony za pomocą transformatora `CombinedAttri`
`↪butesAdder`). Może on w podobny sposób wyszukiwać najlepszą metodę radzenia sobie z elementami odstającymi, brakującymi cechami, doborem cech itd.

W tym przykładzie uzyskujemy najlepsze rozwiązanie, wyznaczając wartość 8 dla hiperparametru `max_features`, a 30 dla hiperparametru `n_estimators`. Błąd RMSE dla tej kombinacji wynosi 49 682, co stanowi nieco lepszy wynik niż w przypadku standardowych wartości tych hiperparametrów (50 182). Gratulacje! Właśnie skutecznie dostroiliśmy nasz najlepszy model!

Metoda losowego przeszukiwania

Mechanizm przeszukiwania siatki przydaje się wtedy, gdy chcemy sprawdzić względnie niewielką liczbę kombinacji (tak jak w powyższym przykładzie), jednak jeśli **przestrzeń przeszukiwania** hiperparametrów jest bardzo duża, lepiej skorzystać z obiektu `RandomizedSearchCV`. Klasa ta jest używana w bardzo podobny sposób, jak `GridSearchCV`, tutaj jednak nie są sprawdzane wszystkie możliwe kombinacje, lecz następuje ewaluacja określonej liczby losowych kombinacji poprzez dobór losowej wartości hiperparametru w każdym przebiegu. Rozwiązanie to cechują dwie zalety:

- Jeśli zezwolisz na, dajmy na to, 1000 przebiegów losowego przeszukiwania, zostanie sprawdzonych 1000 wartości dla każdego hiperparametru (nie jesteśmy ograniczeni wyłącznie do kilku wartości dla każdego hiperparametru, jak w metodzie przeszukiwania siatki).
- Zyskujesz większą kontrolę nad mocą obliczeniową, jaką chcesz przeznaczyć na wyszukiwanie wartości hiperparametrów — wystarczy regulować liczbę przebiegów.

Metody zespołowe

Innym sposobem strojenia modelu jest próba połączenia najlepiej spisujących się modeli. Grupa (lub „zespół”) zawsze ma większą wydajność od jej elementów składowych (tak jak w przypadku modelu losowego lasu składającego się z poszczególnych drzew decyzyjnych), zwłaszcza jeśli poszczególne modele popełniają odmienne rodzaje błędów. Przyjrzymy się temu zagadnieniu dokładniej w rozdziale 7.

Analizuj najlepsze modele i ich błędy

Często uzyskasz wiele informacji na temat problemu, sprawdzając najlepsze modele. Na przykład model `RandomForestRegressor` może wskazywać względną istotność każdego atrybutu w generowaniu dokładnych prognoz:

```
>>> feature_importances = grid_search.best_estimator_.feature_importances_
>>> feature_importances
array([7.33442355e-02,   6.29090705e-02,   4.11437985e-02,
           1.46726854e-02,   1.41064835e-02,   1.48742809e-02,
           1.42575993e-02,   3.66158981e-01,   5.64191792e-02,
           1.08792957e-01,   5.33510773e-02,   1.03114883e-02,
           1.64780994e-01,   6.02803867e-05,   1.96041560e-03,
           2.85647464e-03])
```

Połączmy teraz te wyniki istotności z odpowiadającymi im nazwami atrybutów:

```
>>> extra_attribs = ["Pokoje_na_rodzinę", "Populacja_na_rodzinę", "Sypialnie_na_pokoje"]
>>> cat_encoder = full_pipeline.named_transformers_["cat"]
>>> cat_one_hot_attribs = list(cat_encoder.categories_[0])
>>> attributes = num_attribs + extra_attribs + cat_one_hot_attribs
>>> sorted(zip(feature_importances, attributes), reverse=True)
```

```
[(0.3661589806181342, 'median_income'),
 (0.1647809935615905, 'INLAND'),
 (0.10879295677551573, 'Populacja_na_rodzinę'),
 (0.07334423551601242, 'longitude'),
 (0.0629090704826203, 'latitude'),
 (0.05641917918195401, 'Pokoje_na_rodzinę'),
 (0.05335107734767581, 'Sypialnie_na_pokoje'),
 (0.041143798478729635, 'housing_median_age'),
 (0.014874280890402767, 'population'),
 (0.014672685420543237, 'total_rooms'),
 (0.014257599323407807, 'households'),
 (0.014106483453584102, 'total_bedrooms'),
 (0.010311488326303787, '<1H OCEAN'),
 (0.002856474637320158, 'NEAR OCEAN'),
 (0.00196041559947807, 'NEAR BAY'),
 (6.028038672736599e-05, 'ISLAND')]
```

Dzięki uzyskanym informacjom możesz zrezygnować z niektórych mniej przydatnych cech (przykładowo, wygląda na to, że tylko kategoria `ocean_proximity` jest użyteczna, dlatego warto spróbować usunąć pozostałe).

Przyjrzyj się również błędom popełnianym przez Twój system, następnie postaraj się poznać ich przyczynę oraz znaleźć rozwiązanie (dodać cechy lub usunąć nieprzydatne, pozbyć się elementów odstających itd.).

Oceń system za pomocą zbioru testowego

Po etapie strojenia modeli w końcu uzyskasz system sprawujący się wystarczająco dobrze. Nadszedł czas, aby ocenić jego wydajność za pomocą zbioru danych testowych. Proces ten nie wyróżnia się niczym szczególnym; pobierz przykłady i etykiety z zestawu testowego, wykorzystaj potok `full_pipeline` do przekształcenia tych danych (wywołaj funkcję `transform()`, a *nie* `fit_transform()`: nie chcesz dopasowywać modelu do danych testowych!) i oceń ostateczny model za pomocą tego zbioru testowego.

```
final_model = grid_search.best_estimator_

X_test = strat_test_set.drop("median_house_value", axis=1)
y_test = strat_test_set["median_house_value"].copy()

X_test_prepared = full_pipeline.transform(X_test)

final_predictions = final_model.predict(X_test_prepared)

final_mse = mean_squared_error(y_test, final_predictions)
final_rmse = np.sqrt(final_mse) # => otrzymujemy wynik 47 730,2
```

W pewnych sytuacjach takie punktowe oszacowanie błędu uogólniania może okazać się niewystarczająco przekonujące: a jeżeli model jest zaledwie o 0,1% lepszy od obecnie używanego w środowisku produkcyjnym? Być może chcesz się dowiedzieć, jak precyzyjne jest to oszacowanie. W tym celu możesz obliczyć 95% **przedział ufności** (ang. *confidence interval*) dla błędu uogólniania za pomocą funkcji `scipy.stats.t.interval()`:

```
>>> from scipy import stats
>>> confidence = 0.95
>>> squared_errors = (final_predictions - y_test) ** 2
>>> np.sqrt(stats.t.interval(confidence, len(squared_errors) - 1,
```

```
...                                          loc=squared_errors.mean(),
...                                          scale=stats.sem(squared_errors)))
...
array([45685.10470776, 49691.25001878])
```

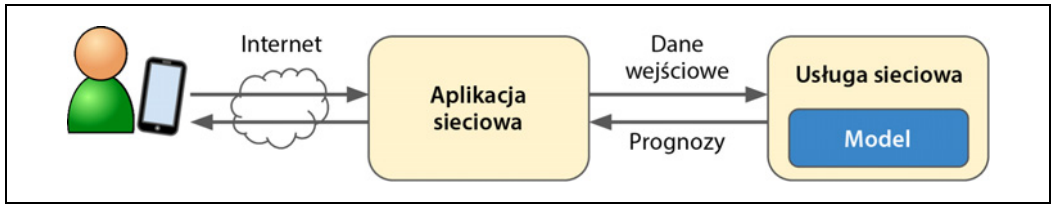
W przypadku intensywnego strojenia hiperparametrów będziemy zazwyczaj otrzymywać nieco gorsze wyniki niż zmierzone za pomocą sprawdzianu krzyżowego (gdyż system zostaje dostrojony do danych walidacyjnych i prawdopodobnie nie będzie sobie tak dobrze radził z nieznanymi próbkami). Akurat tak się nie dzieje w naszym przykładzie, ale jeśli Ci się to przytrafi, musisz powstrzymać chęć dalszego poprawiania hiperparametrów w celu uzyskania dobrych wyników dla zbioru testowego; wszelkie usprawnienia już raczej nie poprawią procesu uogólniania wobec nowych danych.

Teraz następuje faza przeduruchomieniowa projektu: musisz przedstawić rozwiązanie (zwracając szczególną uwagę na to, czego się nauczyłaś/nauczyłeś, co zadziałało, a co nie, jakie przyjęłaś/przyjąłeś założenia, a także zaznaczając ograniczenia systemu), wszystko udokumentować, a także stworzyć eleganckie prezentacje zawierające wyraźne wizualizacje wyników oraz łatwe do zapamiętania stwierdzenia (np. „mediana dochodów stanowi główny predyktor cen mieszkań"). W przykładzie z zestawem danych *Housing* ostateczna wydajność systemu okazuje się nie lepsza od oszacowań ekspertów, które często były nawet o 20% inne od rzeczywistych cen, mimo to jednak warto uruchomić taki model, choćby po to, aby zaoszczędzić czas ekspertów — mogą oni wtedy zająć się ciekawszymi lub bardziej produktywnymi zadaniami.

Uruchom, monitoruj i utrzymuj swój system

Znakomicie! Otrzymaliśmy pozwolenie na uruchomienie systemu! Musisz teraz przygotować nasze rozwiązanie do warunków produkcyjnych (tzn. dopieścić kod, napisać dokumentację i testy itd.). Następnym etapem jest wdrożenie modelu do środowiska produkcyjnego. Jednym ze sposobów jest zapisanie wyuczonego modelu Scikit-Learn (np. za pomocą modułu `joblib`), dołączenie gotowego potoku przetwarzania wstępnego i prognozowania, następnie wczytanie tego wytrenowanego modelu w środowisku produkcyjnym i wykorzystanie go do uzyskiwania predykcji poprzez wywołanie metody `predict()`. Na przykład być może dany model będzie wykorzystywany na stronie internetowej: użytkownik wpisze jakieś dane dotyczące nowego dystryktu, a następnie kliknie przycisk *Oszacuj cenę*. Do serwera sieciowego zostanie wysłane zapytanie zawierające dostarczone dane, skąd trafi do aplikacji sieciowej, a na koniec kod wywoła metodę `predict()` (model powinien zostać wczytany po uruchomieniu serwera, nie chcesz, żeby to się działo za każdym razem, gdy model zostaje użyty). Ewentualnie możesz umieścić model w wyspecjalizowanej usłudze sieciowej, do której aplikacja sieciowa będzie przesyłać zapytania poprzez REST API[25] (rysunek 2.17). W ten sposób ułatwiamy proces aktualizowania modelu do nowych wersji bez konieczności przerywania działania głównej aplikacji. Jednocześnie upraszczamy skalowanie, ponieważ możemy uruchamiać dowolną liczbę usług sieciowych oraz równoważyć obciążenie zapytań przesyłanych przez aplikację sieciową. Ponadto aplikacja sieciowa może umożliwiać korzystanie z innych języków, nie tylko Pythona.

[25] Mówiąc krótko, REST (lub RESTful) API to bazujący na protokole HTTP interfejs API zgodny z pewnymi konwencjami, takimi jak np. wykorzystywanie czasowników HTTP do odczytywania, aktualizowania, tworzenia lub usuwania zasobów (odpowiednio `GET`, `POST`, `PUT` i `DELETE`) oraz definiowanie danych wejściowych i wyjściowych w standardzie JSON.



Rysunek 2.17. Model wdrożony jako usługa sieciowa i wykorzystywany przez aplikację sieciową

Kolejną popularną strategią jest wdrożenie modelu w chmurze, np. serwisie Google Cloud AI Platform (znanym wcześniej jako Google Cloud ML Engine): wystarczy zapisać swój model za pomocą modułu `joblib` i przesłać go do Google Cloud Storage (GCS), następnie przejść do Google Cloud AI Platform i stworzyć nową wersję modelu, wyznaczając plik umieszczony w magazynie GCS. I to wszystko! Uzyskujesz dostęp do prostej usługi sieciowej, która przejmuje zadania równoważenia obciążenia i skalowania. Przyjmuje zapytania JSON zawierające dane wejściowe (np. dotyczące dystryktu) i zwraca odpowiedzi JSON przechowujące prognozy. Możesz następnie użyć takiej usługi sieciowej na swojej własnej stronie internetowej (lub w dowolnym innym środowisku produkcyjnym). Jak się przekonasz w rozdziale 19., wdrażanie modeli TensorFlow w usłudze AI Platform nie różni się zanadto od wdrażania modeli Scikit-Learn.

Jednak nasza praca nie kończy się na wdrażaniu. Musisz także przygotować kod odpowiedzialny za monitorowanie systemu, który będzie sprawdzał działanie w regularnych odstępach czasu i wysyłał powiadomienia w przypadku pojawienia się problemu. Może to być nagła przerwa w działaniu, wynikająca z awarii jakiegoś składnika infrastruktury, lecz również nieznaczny rozkład, niezauważony przez dłuższy czas. Jest to dość powszechny problem, gdyż modele mają tendencję do „rozkładu" wraz z ewoluowaniem danych: istotnie, świat ciągle ulega zmianom, dlatego jeżeli model został wyuczony za pomocą zeszłorocznych danych, może nie być dostosowany do dzisiejszych.



Nawet model wyuczony do klasyfikowania zdjęć psów i kotów może wymagać regularnego trenowania, nie z powodu nagłego zmutowania tych zwierząt, lecz ze względu na rozwój aparatów fotograficznych, a także formatów danych czy współczynników ostrości, jasności i rozmiaru. Ponadto w następnym roku może zapanować moda na inną rasę lub ludzie zaczną zakładać pupilkom kapelusze. Kto wie?

Musisz więc monitorować bieżącą wydajność modelu. Jak to jednak robić? To zależy. W niektórych przypadkach skuteczność modelu można wywnioskować bezpośrednio ze wskaźników cyklu pracy. Na przykład jeżeli Twój model stanowi część systemu rekomendacji i sugeruje produkty, które mogą zainteresować użytkowników, to z łatwością można monitorować liczbę sprzedanych zalecanych produktów. Jeżeli liczba ta zmaleje (w stosunku do produktów niezalecanych), to model staje się głównym podejrzanym. Być może potok danych uległ załamaniu albo należy wyuczyć model za pomocą świeżych danych (wkrótce wrócę do tego tematu).

Nie zawsze jednak jest możliwa ocena wydajności modelu bez analizy wykonanej przez człowieka. Załóżmy na przykład, że wyuczyliśmy model klasyfikujący obrazy (zob. rozdział 3.) do wykrywania kilku usterek produktów na linii montażowej. W jaki sposób zostaniesz zaalarmowana/zaalarmowany o spadku skuteczności modelu, zanim tysiące wadliwych produktów trafi do klientów? Jednym ze sposobów jest przesłanie do ludzkich kontrolerów jakości części obrazów sklasyfikowanych

przez model (zwłaszcza tych, przy których nie miał dużej pewności). W zależności od zadania kontrolerzy mogą być ekspertami lub laikami, np. pracownikami źródeł społecznościowych (takich jak Amazon Mechanical Turk). W pewnych sytuacjach mogą być oni nawet samymi użytkownikami, odpowiadającymi na przykład za pomocą ankiet lub zmodyfikowanych testów captcha[26].

Tak czy inaczej musisz ustanowić system monitorujący (uwzględniający ludzkich kontrolerów kontrolujących pracujący model, ale niekoniecznie), a także wszelkie istotne procesy definiujące zachowanie w przypadku wystąpienia awarii i sposoby przygotowania się do nich. Niestety zadanie to jest pracochłonne. Tak naprawdę często wymaga poświęcenia więcej czasu niż w przypadku stworzenia i wytrenowania modelu.

Jeżeli dane ciągle ewoluują, musisz zaktualizować swoje zestawy danych i regularnie trenować model. W miarę możliwości zautomatyzuj cały proces. Oto kilka elementów, które można zautomatyzować:

- Regularne gromadzenie i oznaczanie świeżych danych (np. za pomocą ludzkich kontrolerów).
- Stworzenie skryptu automatycznie uczącego model i dostrajającego hiperparametry. Skrypt ten może być uruchamiany automatycznie, np. codziennie albo co tydzień, w zależności od zapotrzebowania.
- Napisanie kolejnego skryptu, który ocenia wydajność zarówno nowego, jak i starego modelu za pomocą zaktualizowanego zestawu testowego, a także wdraża model do środowiska produkcyjnego, jeżeli jego skuteczność nie zmalała (jeżeli zmalała, to musisz znaleźć przyczynę).

Musisz także ocenić jakość danych wejściowych. Czasami wydajność nieznacznie spada z powodu słabej jakości sygnału (np. pochodzącego z uszkodzonego czujnika przesyłającego losowe wartości lub niewielkiej dynamiki sygnałów wysyłanych przez zespół znajdujący się na wcześniejszym etapie potoku), jednak może minąć trochę czasu, zanim jakość systemu spadnie na tyle, żeby zostało wysłane powiadomienie. Jeśli będziesz obserwować dane wejściowe, możesz wykryć problem nieco wcześniej. Możesz przykładowo wprowadzić alert ostrzegający przed coraz większą liczbą danych wejściowych, w których brakuje jakiejś cechy albo średnia lub odchylenie standardowe zbytnio odbiegają od zestawu testowego, albo w cesze kategorialnej zaczynają pojawiać się nowe kategorie.

Na koniec nie zapomnij o przygotowaniu kopii zapasowych każdego modelu, a także o wprowadzeniu procesu i narzędzi umożliwiających szybkie przywrócenie modelu z kopii zapasowej na wypadek, gdyby nowy model z jakiejś przyczyny zaczął zachowywać się nieprawidłowo. Dzięki kopiom zapasowym możesz również porównywać modele nowe z wcześniejszymi. To samo dotyczy kopii zapasowych zestawów danych, gdyż dzięki temu można cofnąć się do wcześniejszego zestawu danych, gdyby nowy zestaw został uszkodzony (np. gdyby w dodawanych świeżych danych występowało zbyt dużo elementów odstających). Za pomocą kopii zestawów danych możesz także oceniać każdy model za pomocą dowolnej wersji zestawu danych.

Jak widać, na uczenie maszynowe składa się całkiem rozbudowana infrastruktura, dlatego nie zdziw się, jeżeli stworzenie i wdrożenie Twojego pierwszego projekt UM będzie wymagało mnóstwa czasu i wysiłku. Na szczęście po przygotowaniu infrastruktury przejście od pomysłu do produkcji stanie się znacznie szybsze.

[26] Captcha to test sprawdzający, czy użytkownik jest robotem. Testy te są często używane jako tani sposób oznaczania danych uczących.



Możesz chcieć stworzyć kilka podzbiorów zestawu testowego, aby ocenić skuteczność modelu w określonych obszarach danych. Na przykład możesz stworzyć podzbiór zawierający wyłącznie najświeższe dane lub zestaw testowy przeznaczony dla określonych rodzajów danych wejściowych (np. dystrykty znajdujące się w głębi lądu i dystrykty mieszczące się nad oceanem). W ten sposób łatwiej odkryjesz mocne i słabe strony modelu.

Teraz Twoja kolej!

Mam nadzieję, że dzięki niniejszemu rozdziałowi wiesz już mniej więcej, jak wygląda typowy projekt uczenia maszynowego, oraz że przydadzą Ci się omówione narzędzia, za pomocą których jesteś w stanie stworzyć wspaniały system. Jak widać, lwią część czasu pochłaniają etapy przygotowywania danych, tworzenia narzędzi monitorujących, konfigurowania potoków oceniania wydajności oraz automatyzowania regularnego uczenia modelu. Oczywiście algorytmy uczenia maszynowego są istotne, jednak bardziej opłaca się zaznajomić się z całym procesem przygotowywania projektu i dobrze opanować trzy lub cztery algorytmy, niż poświęcać cały swój czas na poznawanie zaawansowanych algorytmów.

Jeśli więc jeszcze tego nie zrobiłaś/zrobiłeś, teraz nadeszła dobra pora na włączenie laptopa, wybranie interesującego Cię zbioru danych i przebrnięcie przez cały omawiany proces od początku do końca. Warto rozpocząć swoją przygodę od strony *https://www.kaggle.com/*. Znajdziesz tam odpowiedni zbiór danych dla siebie, wyraźny cel oraz osoby, z którymi możesz się dzielić doświadczeniami. Baw się dobrze!

Ćwiczenia

Poniższe ćwiczenia bazują na omówionym w tym rozdziale zestawie danych *Housing*:

1. Wypróbuj regresor maszyny wektorów nośnych (`sklearn.svm.SVR`) przy użyciu różnych hiperparametrów, takich jak `kernel="linear"` (oraz różnych wartości hiperparametru `C`) lub `kernel="rbf"` (oraz różnych wartości hiperparametrów `C` i `gamma`). Na razie nie przejmuj się tym, że nie wiesz, do czego te hiperparametry służą. Jak się spisuje najlepszy predyktor maszyny wektorów nośnych?
2. Spróbuj zastąpić klasę `GridSearchCV` obiektem `RandomizedSearchCV`.
3. Spróbuj dodać w potoku przygotowawczym funkcję przekształcającą w taki sposób, aby były dobierane wyłącznie najistotniejsze atrybuty.
4. Spróbuj stworzyć pojedynczy potok przeprowadzający pełne przygotowywanie danych i generujący ostateczne prognozy.
5. Sprawdź automatycznie niektóre funkcje przygotowawcze za pomocą klasy `GridSearchCV`.

Rozwiązania tych ćwiczeń znajdziesz w notatniku Jupyter dostępnym pod adresem *ftp://ftp.helion.pl/przyklady/uczem2.zip*.